

Machine Learning

Como as máquinas aprendem com dados, do supervisionado ao não-supervisionado

Prof. Me. Otávio Parraga

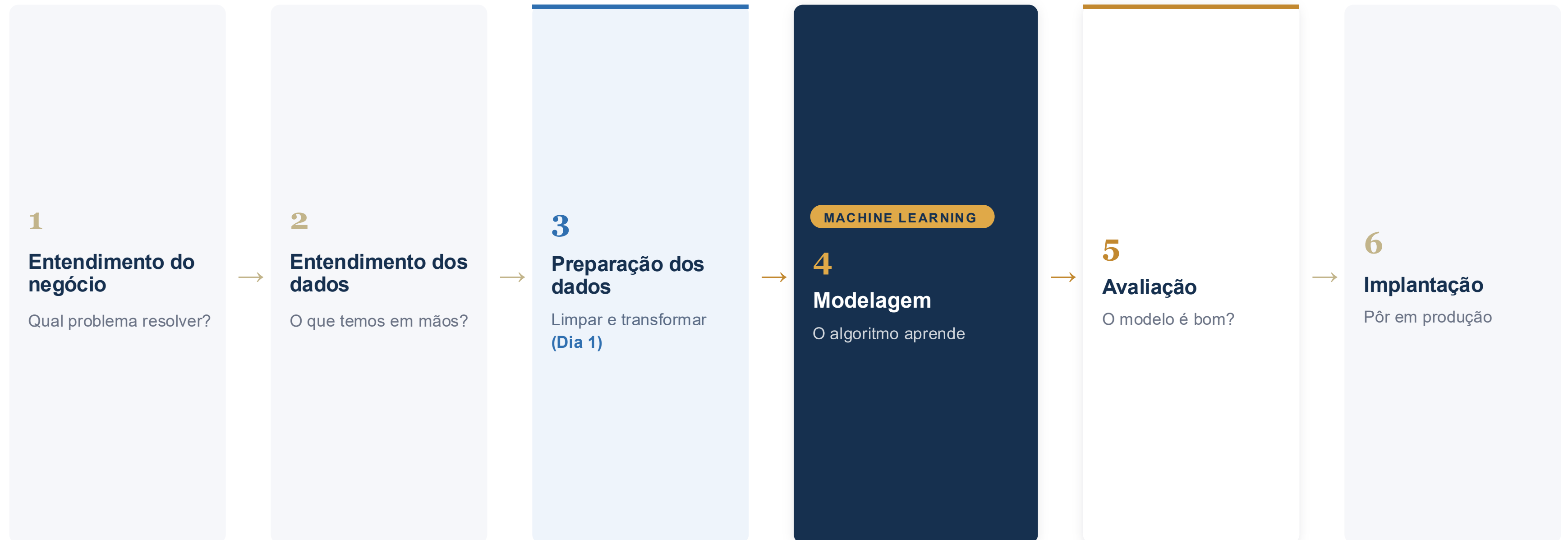
Bacharelado em Ciência de Dados e Inteligência Artificial

MALTA
Machine Learning Theory
and Applications Lab



↺ processo iterativo

Onde entra o Machine Learning



Antes de modelar: preparar os dados

Os dados quase nunca chegam prontos. O **pré-processamento** reúne duas grandes famílias de operações, e costuma consumir a maior parte do tempo de um projeto.

1 · Limpeza

Corrigir problemas de **qualidade** dos dados.

- Valores **ausentes**
- Valores **duplicados**
- **Outliers** e inconsistências

2 · Transformação

Adaptar os dados aos **algoritmos**.

- **Normalização** e padronização
- **Codificação** de variáveis categóricas
- **Feature engineering**

LEMBRE-SE *Garbage in, garbage out*

O pipeline do aprendizado supervisionado

Aprender com **exemplos rotulados** ($X \rightarrow y$): o caminho de hoje, da base de dados ao modelo em produção — passando por **treinar** e **avaliar**.



TREINAR = APRENDER COM EXEMPLOS

O modelo vê **entradas e respostas conhecidas** e ajusta seus parâmetros para **errar menos**. É a etapa em que ele "aprende os padrões" dos dados.

AVALIAR = TESTAR NO QUE NUNCA VIU

Medimos o desempenho em **dados separados (teste)**. Se vai bem ali, tende a **generalizar**; se não, voltamos a ajustar.

A trilha de hoje

01 **O que é Machine Learning**
Definição · tipos de aprendizado · fluxo clássico · modelo × algoritmo

02 **Aprendizado Supervisionado**
k-NN · árvore de decisão · regressões · bias-variance · overfitting

03 **Avaliação e Validação**
treino/teste · validação cruzada · matriz de confusão · métricas · ROC

04 **Aprendizado Não-Supervisionado**
k-Means · hierárquico · regras de associação

05 **Redes Neurais e Deep Learning**
neurônio artificial · redes profundas · CNNs · Transformers · aplicações

MÓDULO 01

O que é Machine Learning

Definição formal, tipos de aprendizado, o fluxo clássico de um projeto e a diferença entre modelo e algoritmo.

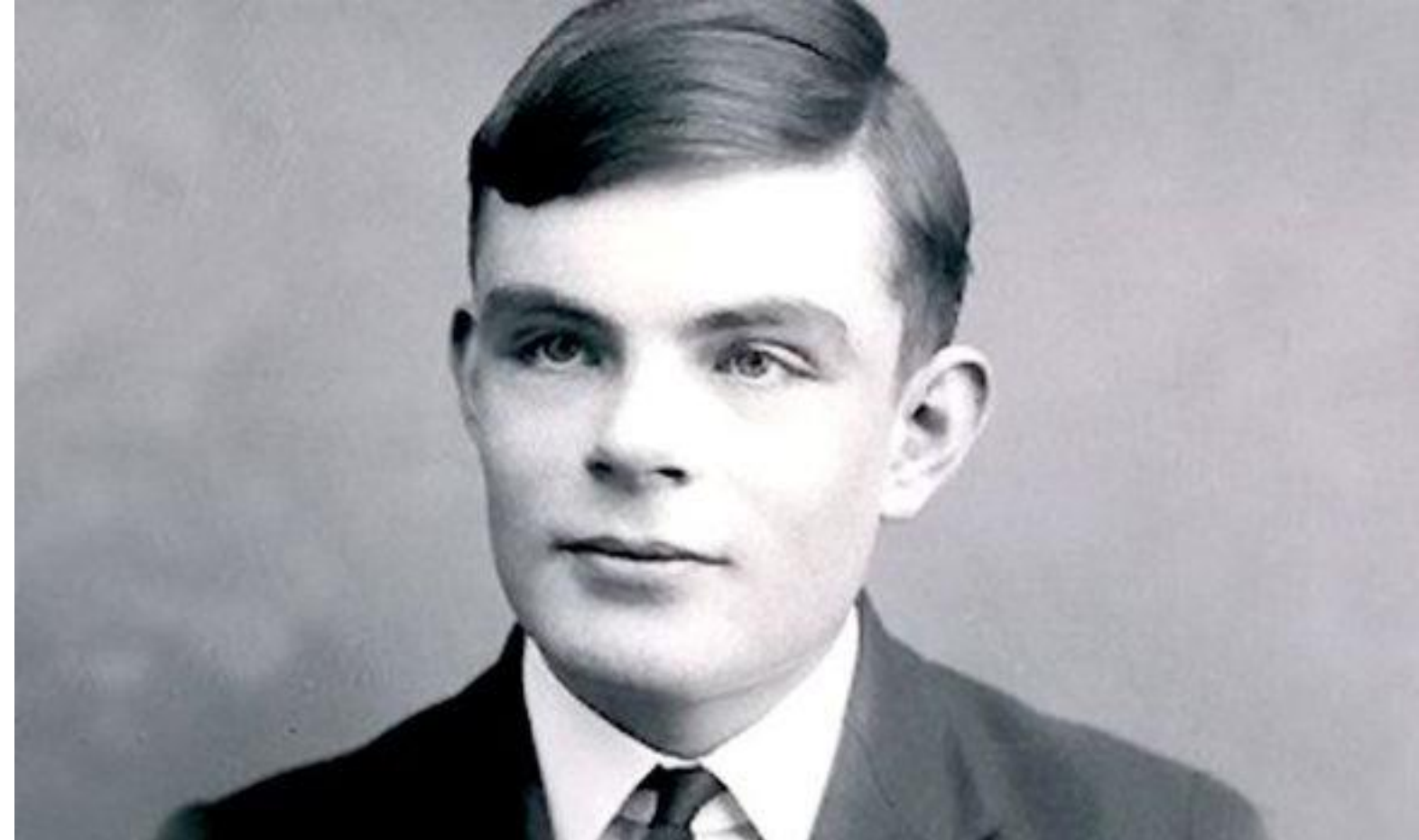
01

Machine Learning não é algo novo

A. M. Turing (1950) Computing Machinery and Intelligence. *Mind* 49: 433-460.

COMPUTING MACHINERY AND INTELLIGENCE

By A. M. Turing

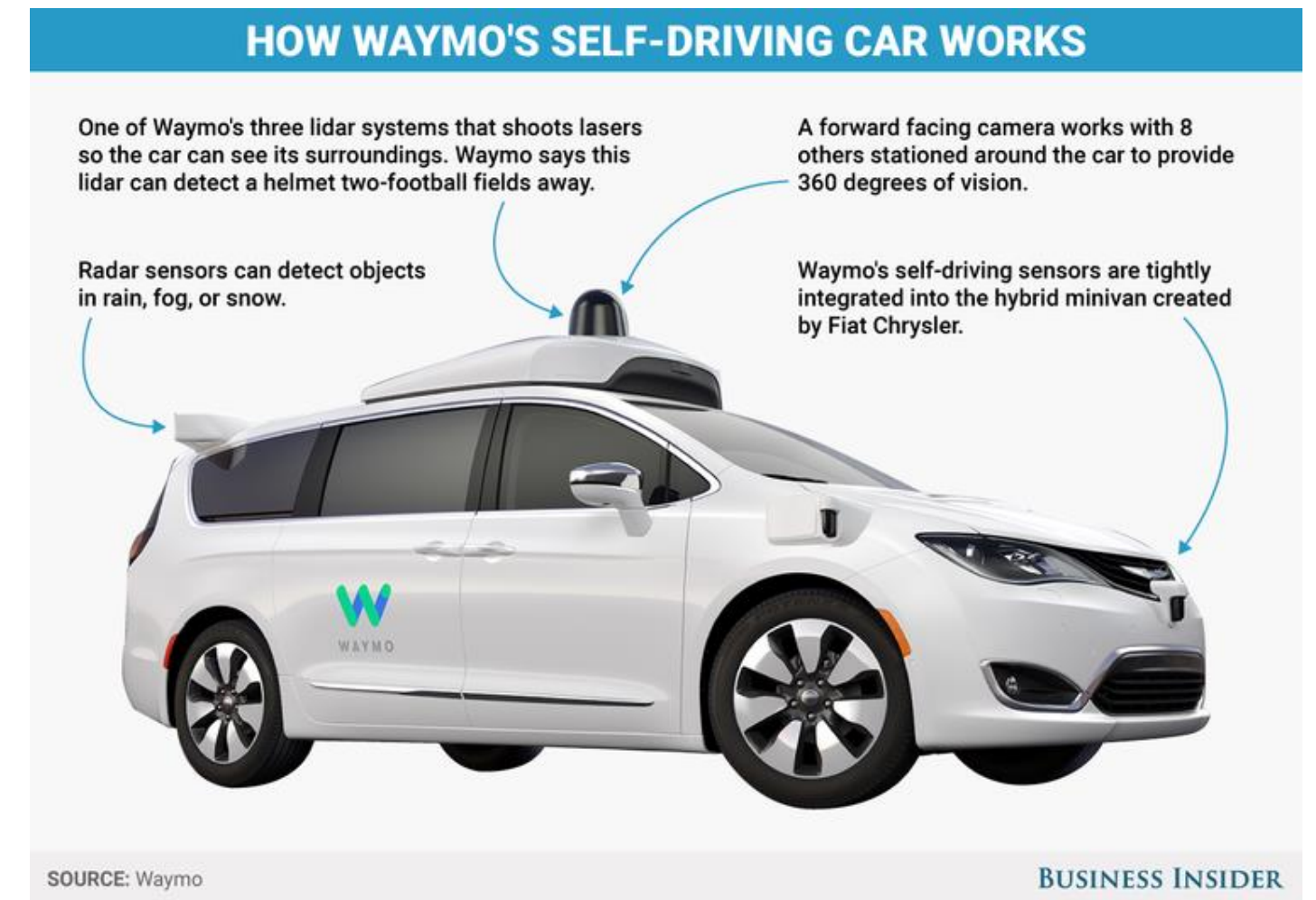
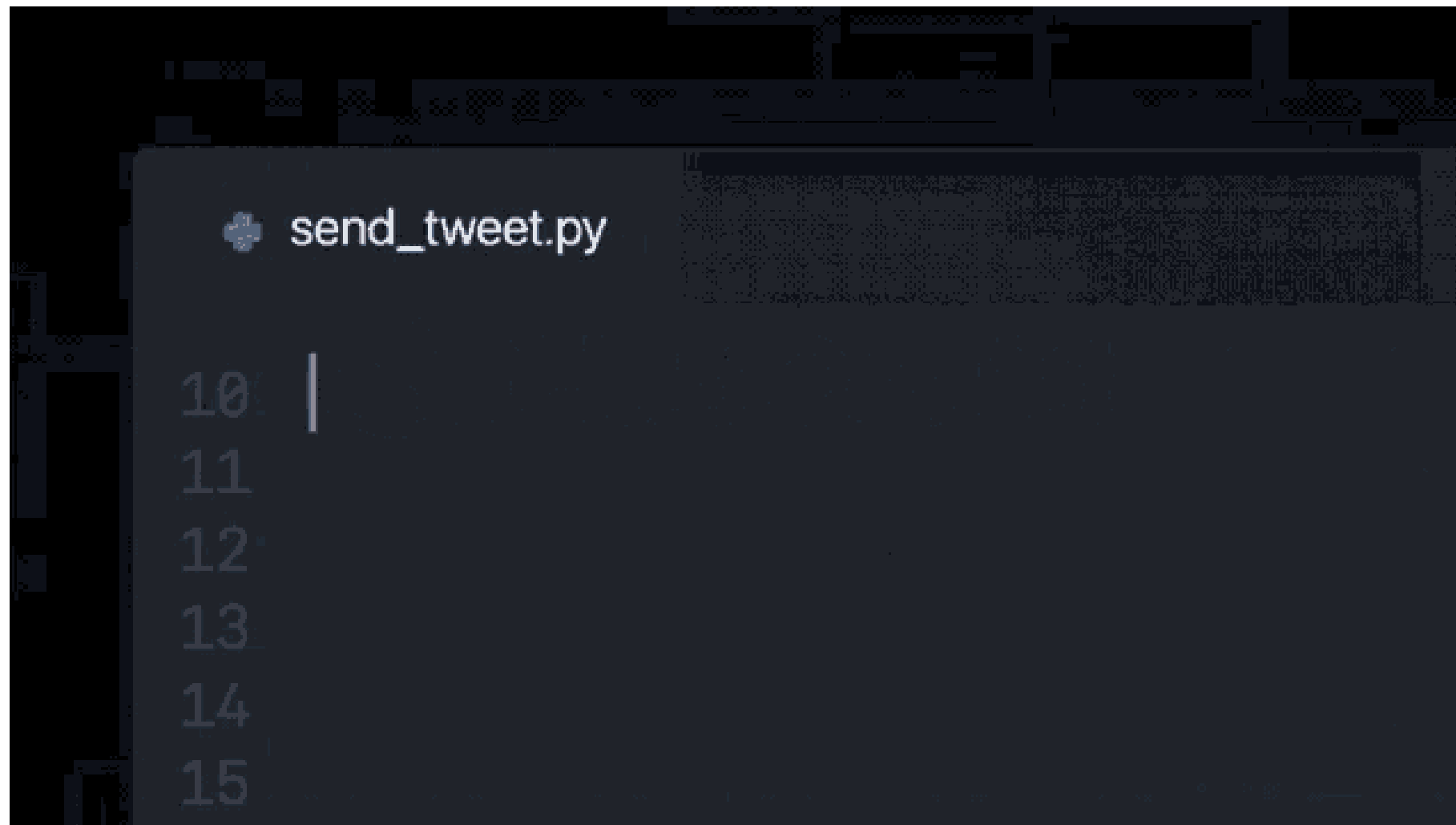


Machine Learning já está em todo lugar

“Na medida que os computadores se tornam mais sofisticados, parece inevitável que Aprendizado de Máquina exerça um papel central em Ciência da Computação e tecnologia de computadores.”

Tom Mitchell

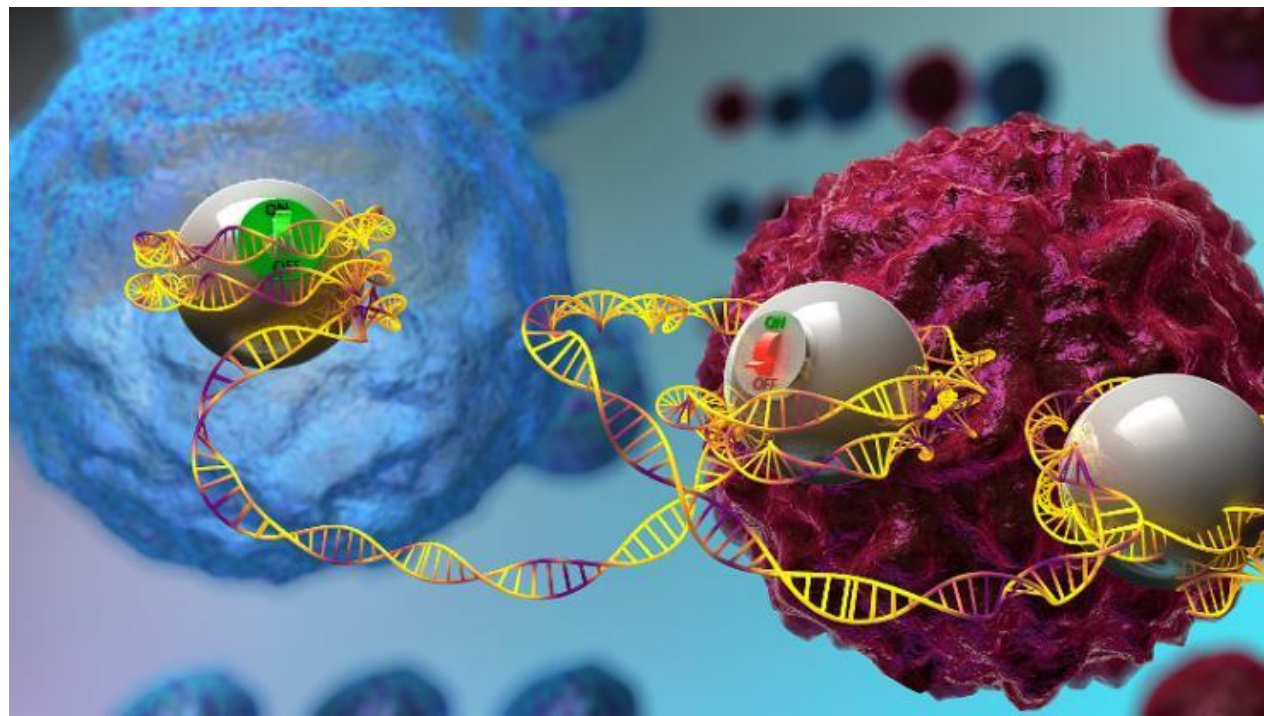
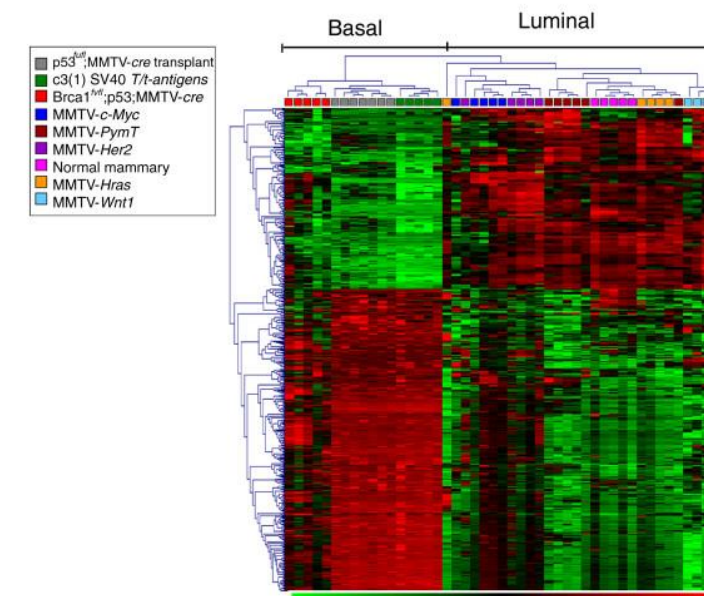
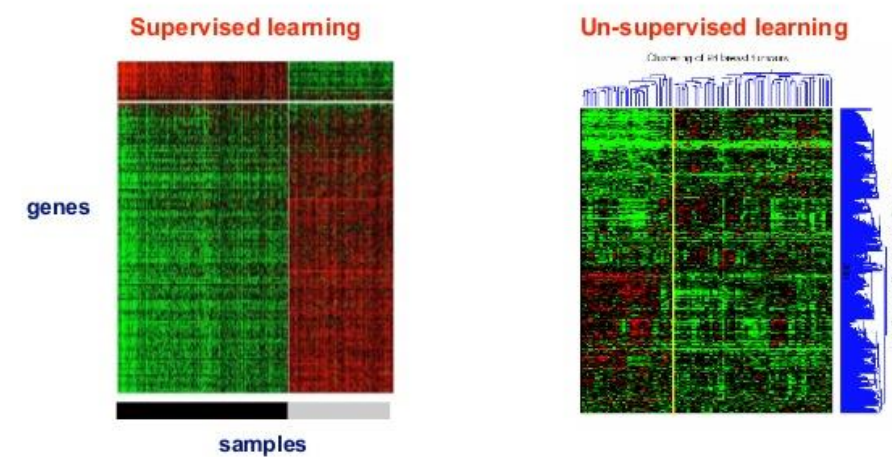
Machine Learning já está em todo lugar



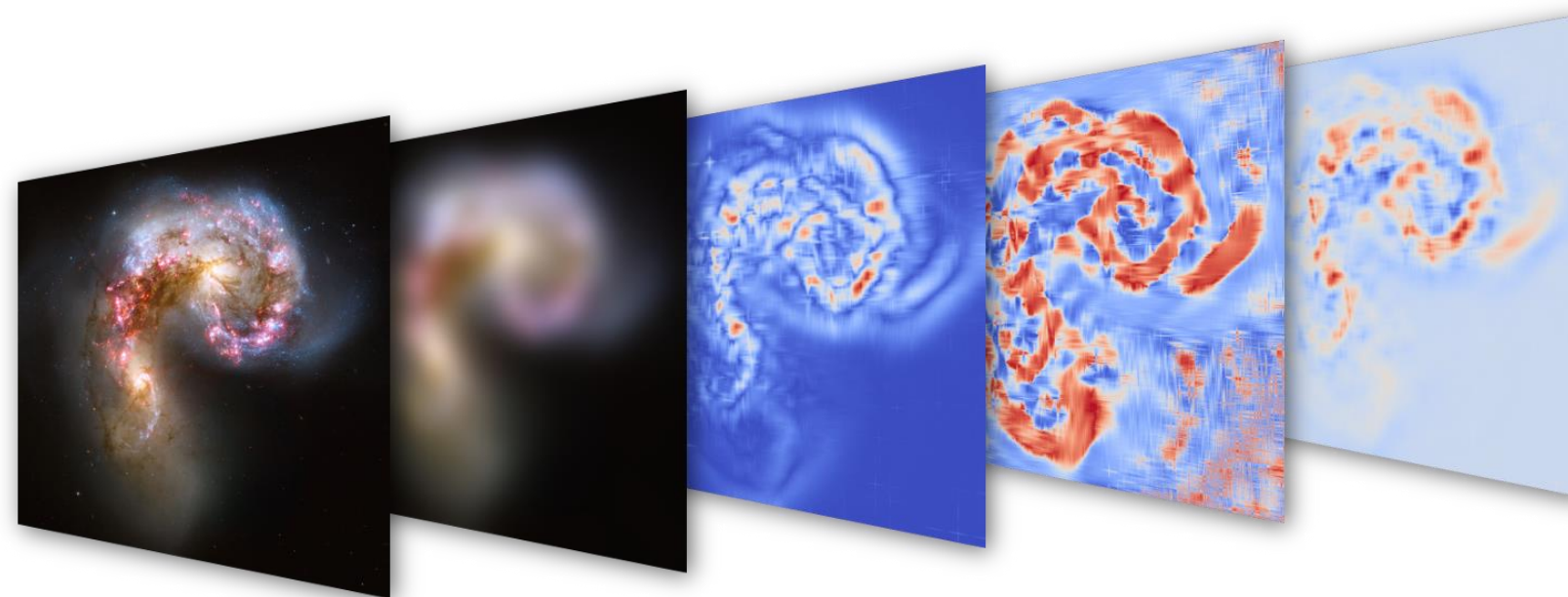
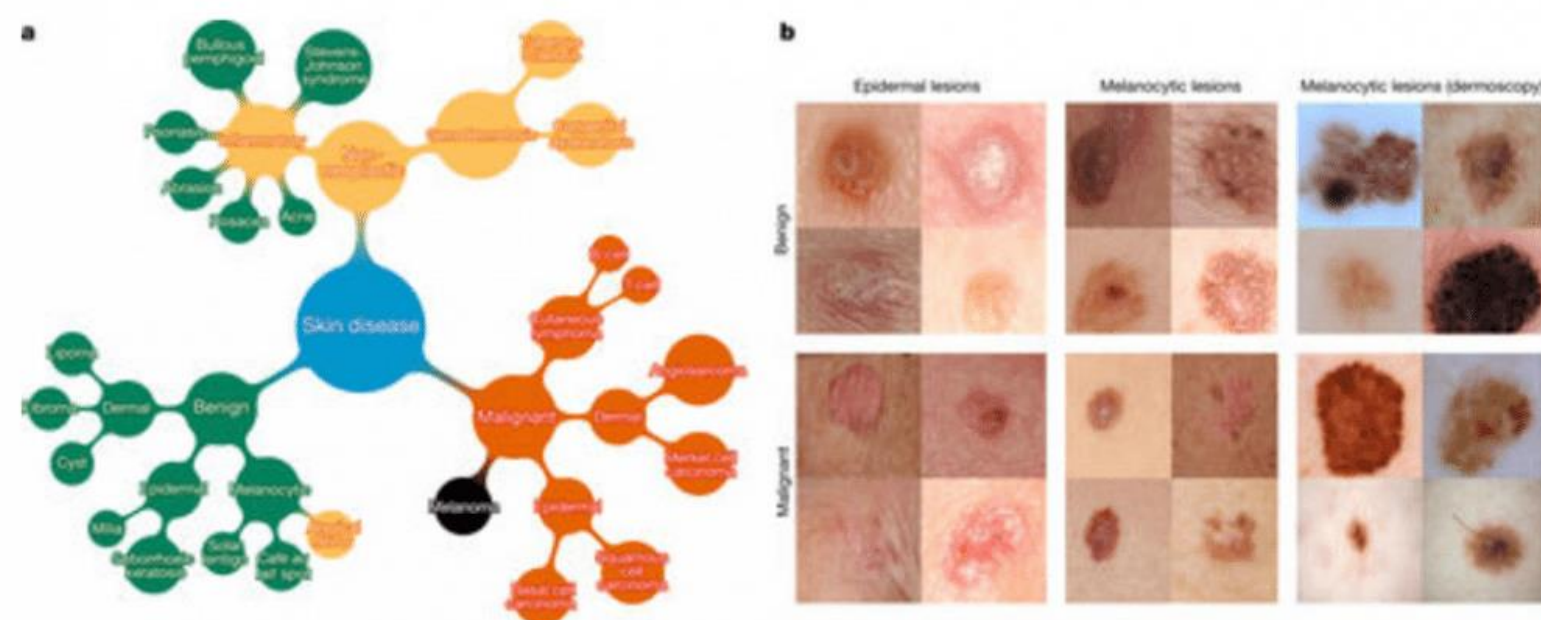
Aplicações de AM

- Algoritmos de AM têm sido bem sucedidos para:
 - Identificar genes associados a determinadas doenças

Analyzing gene expression data

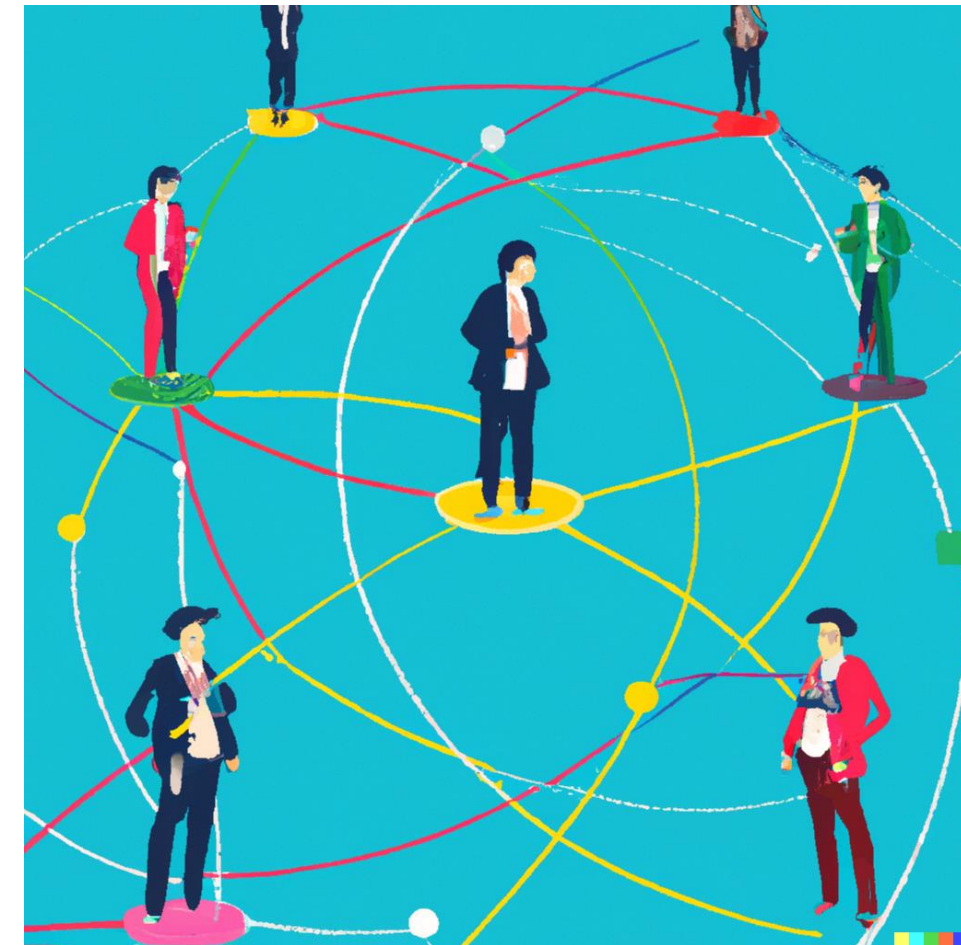


- Discriminar tecidos (saudáveis/doentes), objetos celestiais, ...



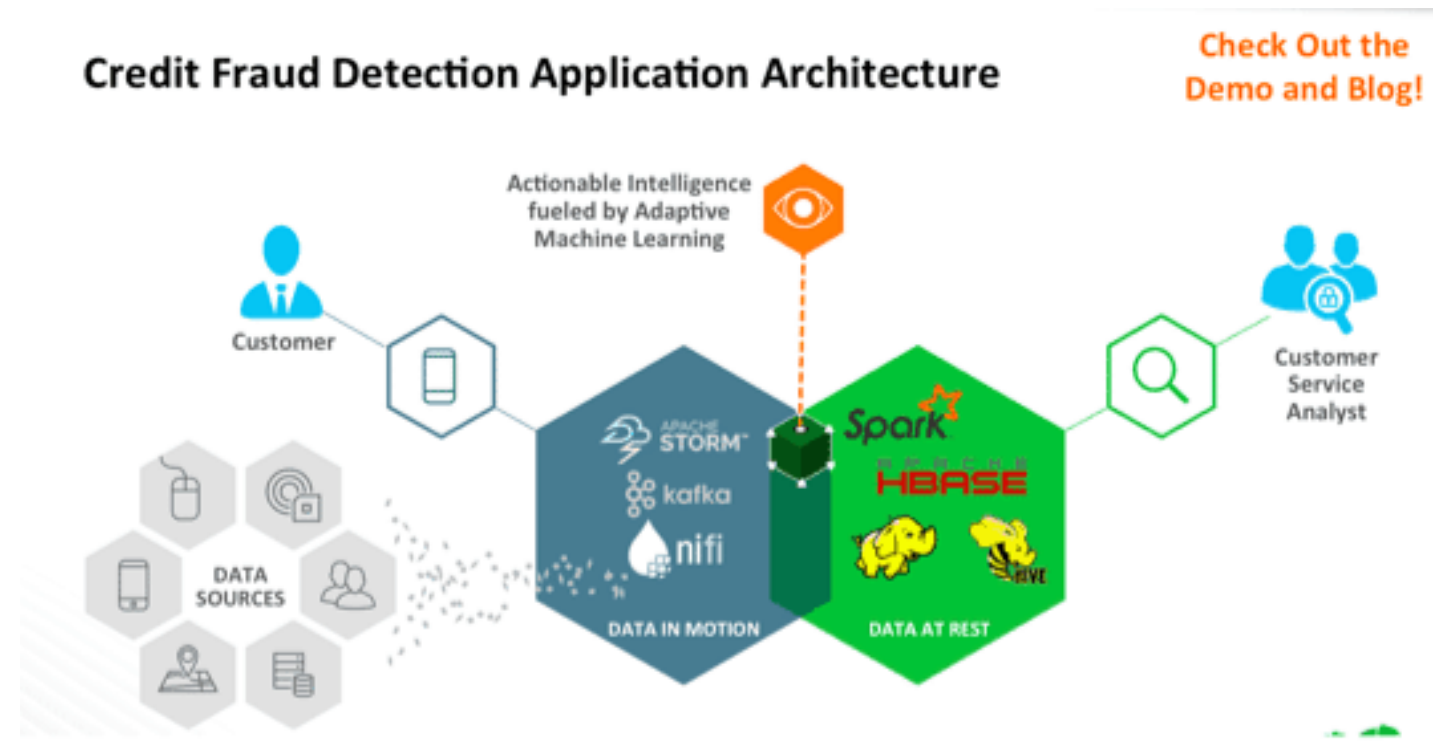
Aplicações de AM

- **Algoritmos de AM têm sido bem sucedidos para:**
 - Identificar nichos de mercado



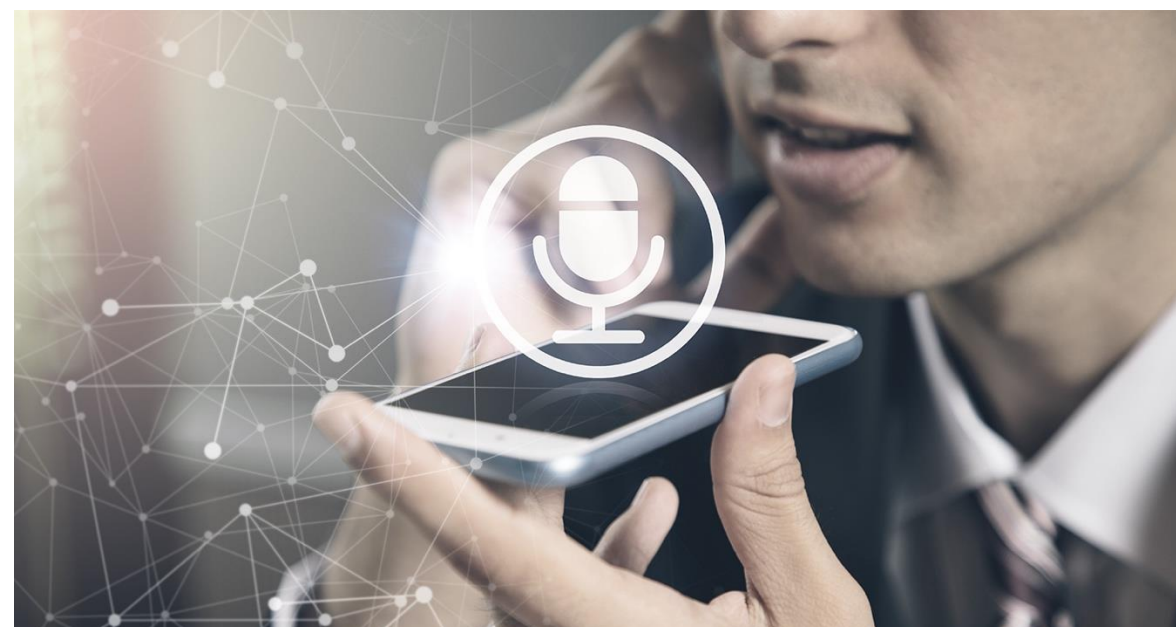
Aplicações de AM

- Algoritmos de AM têm sido bem sucedidos para:
 - Detectar uso fraudulento de cartões de crédito



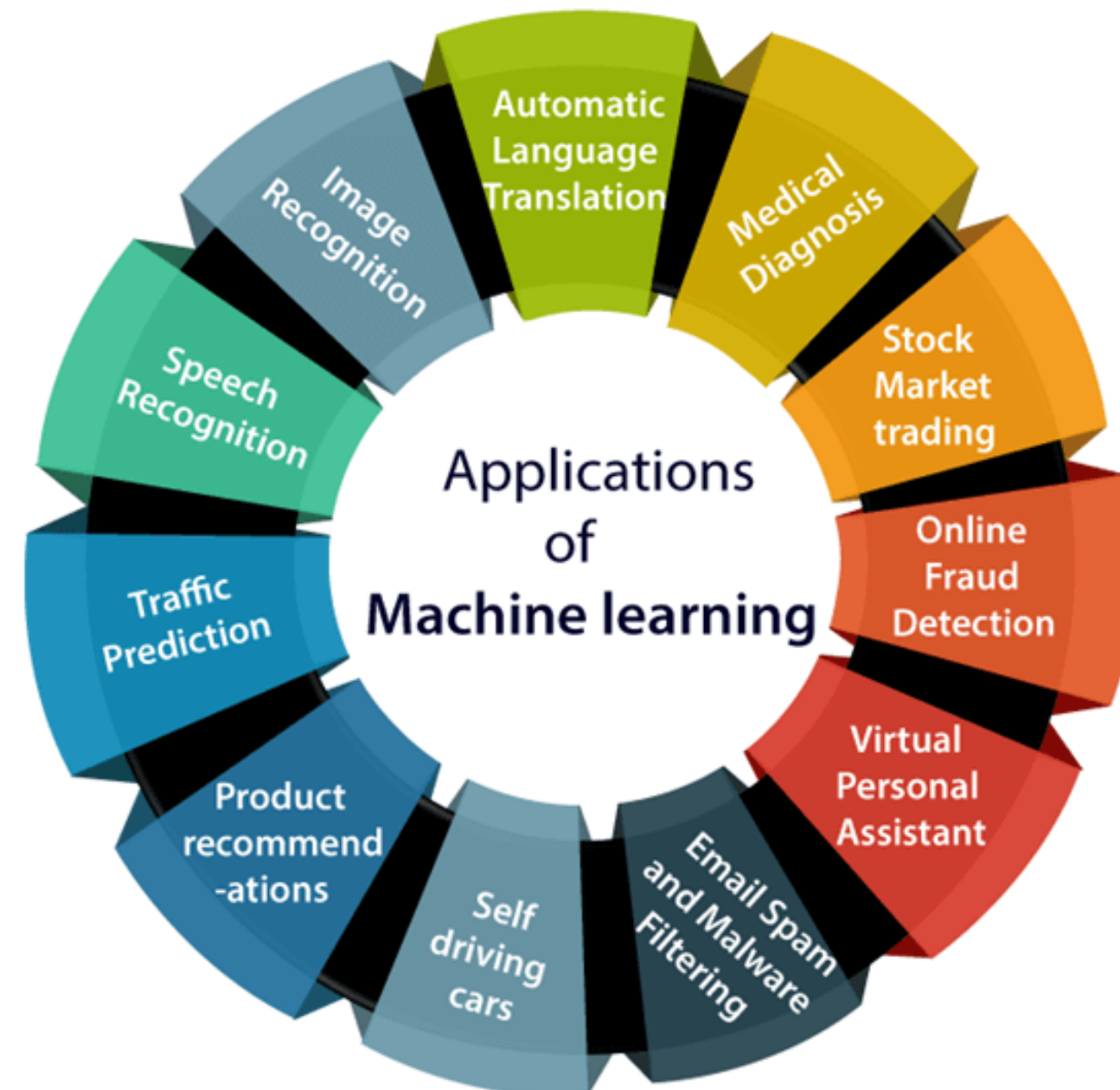
Aplicações de AM

- **Algoritmos de AM têm sido bem sucedidos para:**
 - Reconhecimento facial, de voz, de assinaturas



Aplicações de AM

- **Algoritmos de AM têm sido bem sucedidos para:**
 - e tantas (!!!) outras aplicações



Exemplo: ALVINN



Dean Pomerleau (1989)
CMU

ALVINN

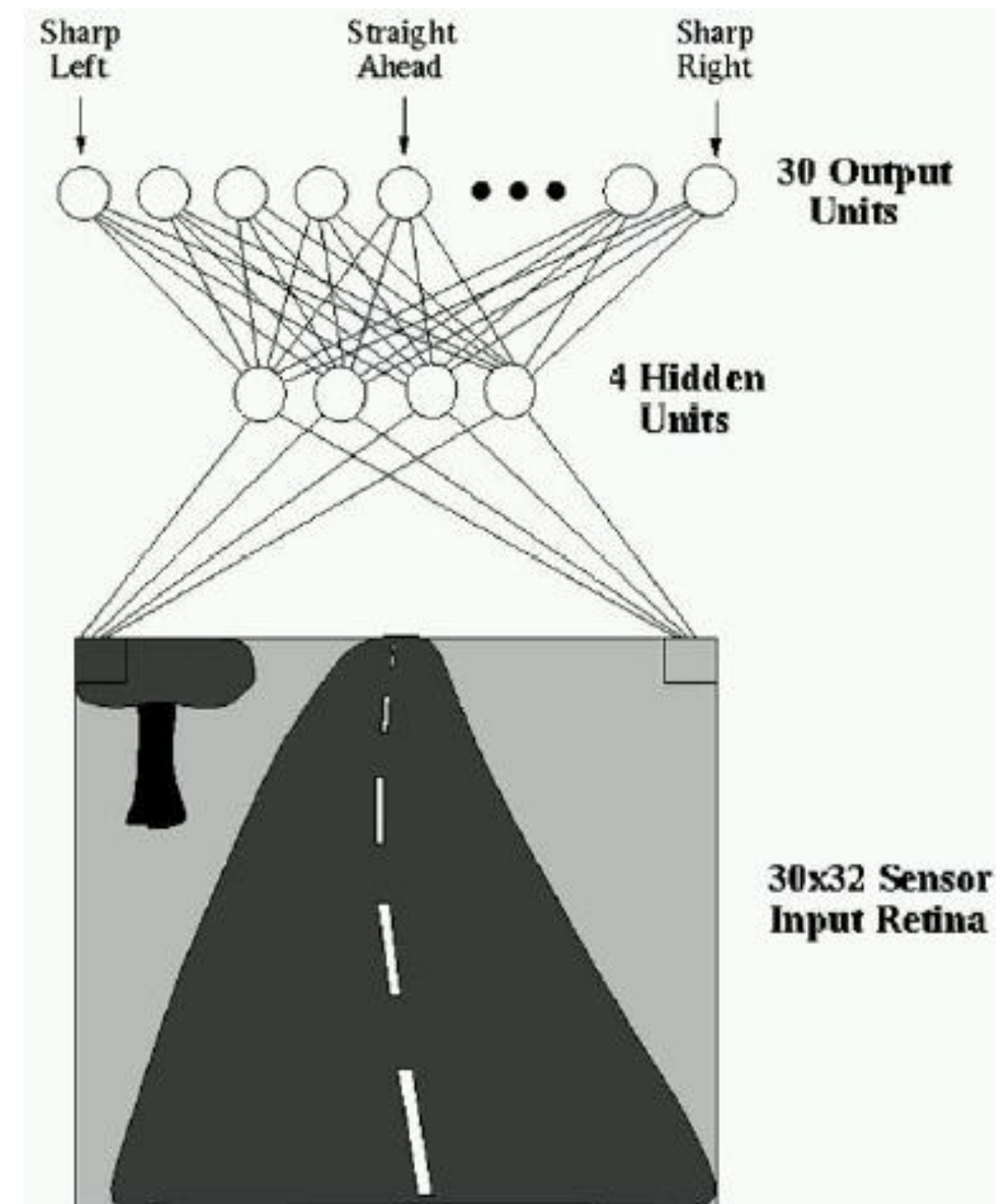
- **Sistema automático de navegação para automóveis**
 - Baseado em imagens de uma câmera *onboard*
 - Dirigiu a 110km/h em rodovia pública americana
 - De costa a costa em 1989 por 2850 milhas



ALVINN

- **Utiliza uma Rede Neural**

- 960 entradas
 - Matrix 30x32 derivada dos pixels da imagem
- 4 neurônios intermediários
- 30 neurônios de saída
 - Cada um representando um comando de direção



Machine Learning já está em todo lugar

Recomendação

Filmes, músicas e produtos sugeridos a partir do seu histórico.

Detecção de fraude

Transações suspeitas sinalizadas em tempo real.

Visão computacional

Reconhecer rostos, ler placas, apoiar diagnósticos por imagem.

Linguagem natural

Tradução, busca, assistentes e filtros de spam.

Previsão de demanda

Estoque, preços e logística antecipados por dados.

Saúde

Estimar risco de doenças e personalizar tratamentos.

O que é aprender, formalmente

*"Diz-se que um programa aprende com a experiência **E**, em relação a uma classe de tarefas **T** e uma medida de desempenho **P**, se seu desempenho em **T**, medido por **P**, melhora com a experiência **E**."*

Tom Mitchell, 1997

T

Tarefa

Classificar e-mails em **spam** ou **não-spam**.

E

Experiência

Milhares de e-mails já **rotulados** como spam ou não.

P

Desempenho

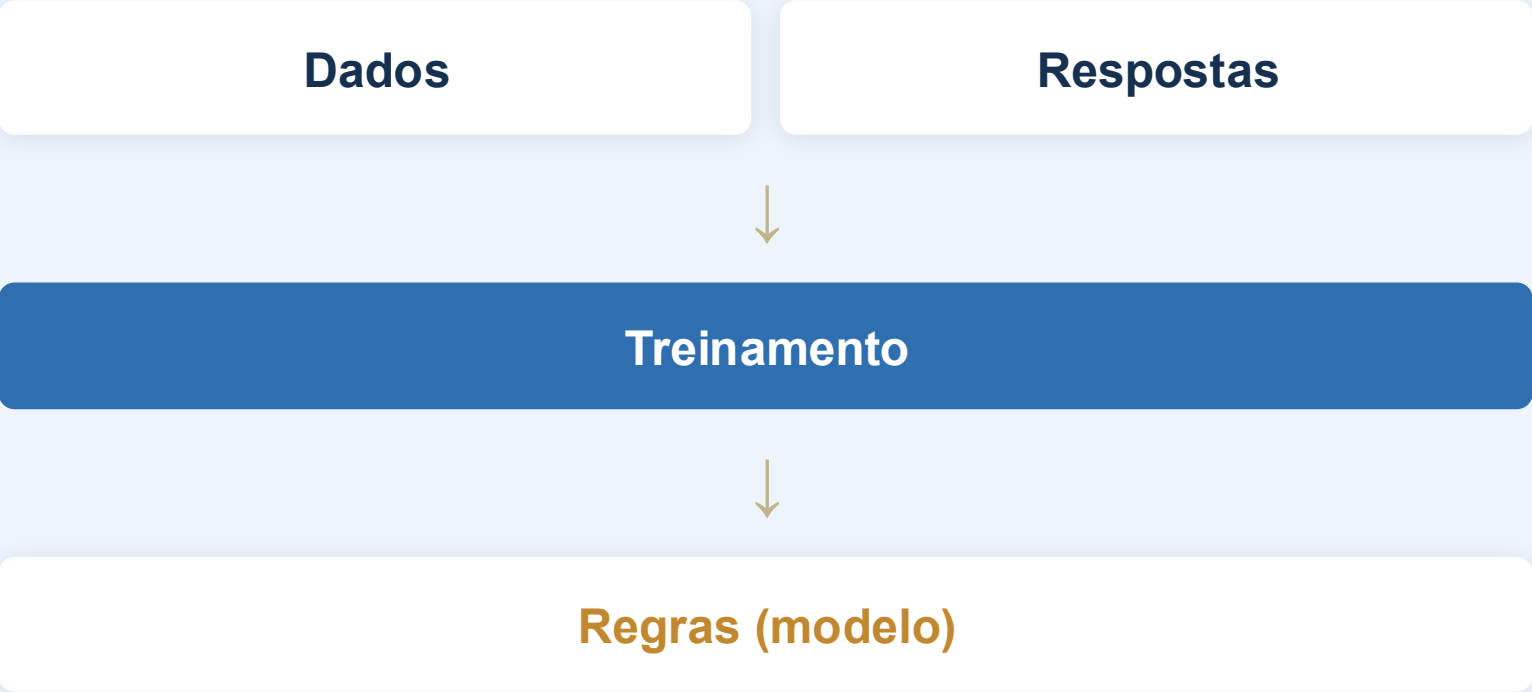
A **proporção** de e-mails classificados corretamente.

ML × programação tradicional

PROGRAMAÇÃO TRADICIONAL



MACHINE LEARNING



Quatro tipos de aprendizado

Supervisionado

Dados **rotulados** (entrada → resposta). Aprende a prever o rótulo. Ex.: classificação, regressão.

Não-supervisionado

Dados **sem rótulo**. Encontra estrutura escondida. Ex.: clustering, PCA.

Semi-supervisionado

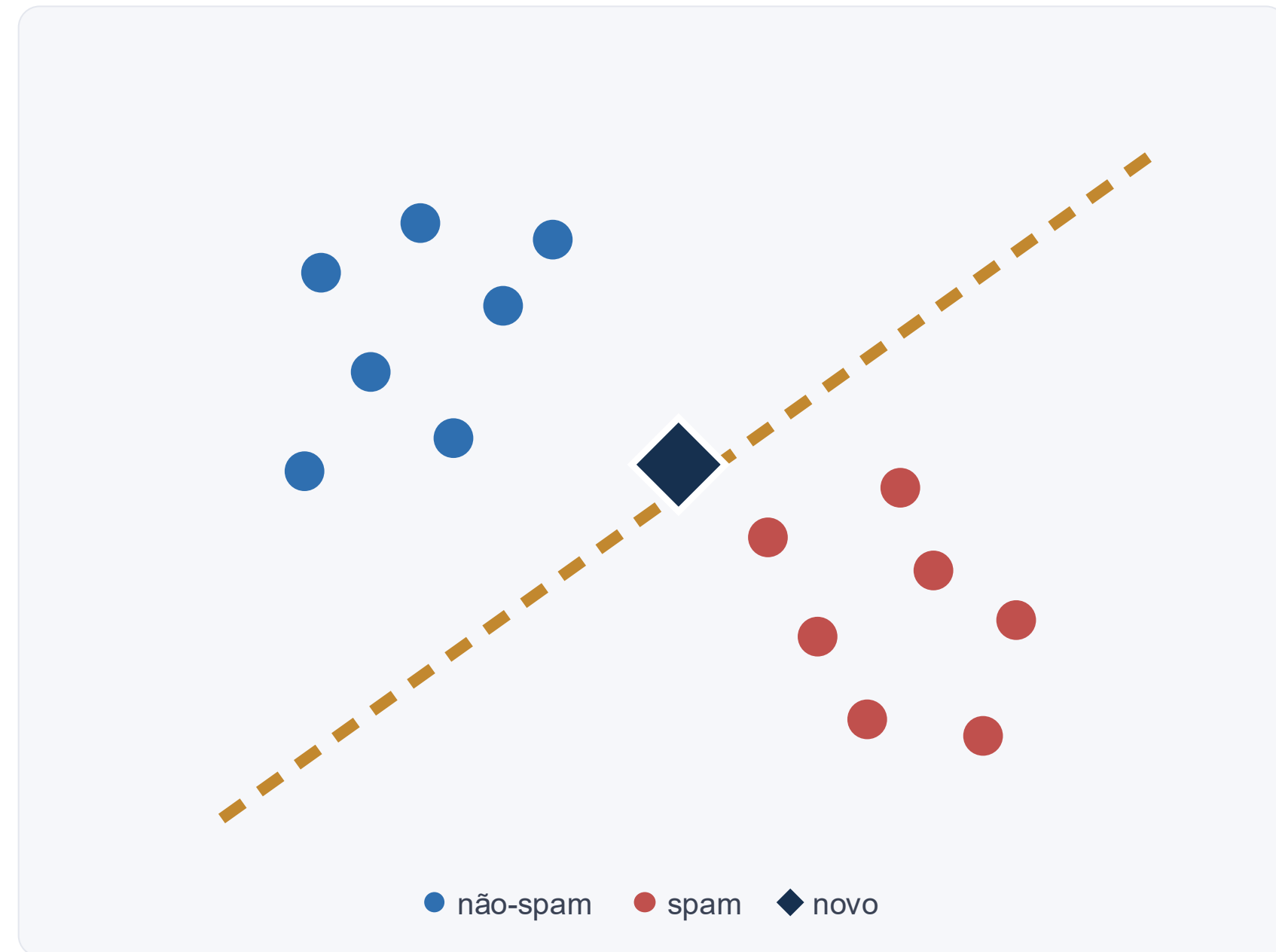
Poucos rótulos + muitos dados não rotulados. Rotular é caro. Ex.: classificar imagens médicas.

Por reforço

Um **agente** aprende por tentativa e erro, buscando **recompensas**. Ex.: jogos, robótica.

Aprendizado supervisionado

Aprende com **exemplos rotulados** (entrada → resposta) e passa a **prever o rótulo** de casos novos.



E-mails **rotulados** → o modelo aprende a **fronteira** que separa as classes

■ Cada exemplo traz a **resposta certa**; o erro guia o ajuste do modelo.

TAREFAS TÍPICAS

Classificação

Prever uma **categoria**. Spam × não-spam.

Regressão

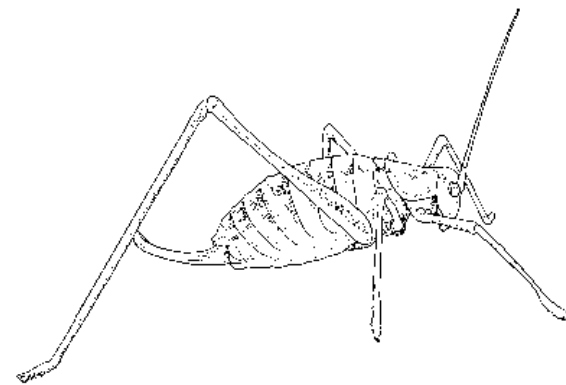
Prever um **valor** contínuo. Preço de um imóvel.

O problema de classificação

(definição informal)

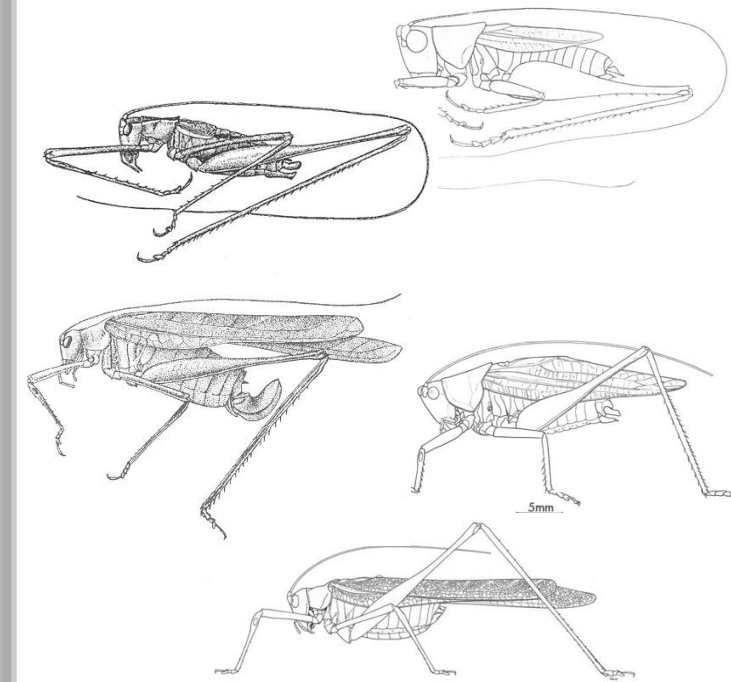
- Dada uma coleção de dados detalhados (neste caso 5 exemplos de **Esperança** e 5 de **Gafanhoto**), decida a qual tipo de inseto o exemplo não rotulado abaixo pertence:

Obs: **Esperança** = tipo de gafanhoto verde

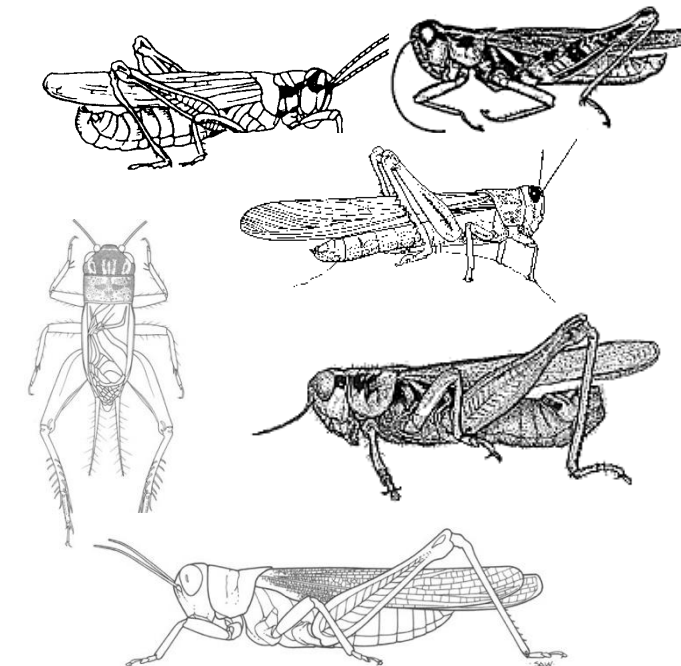


Esperança ou **Gafanhoto**?

Esperança



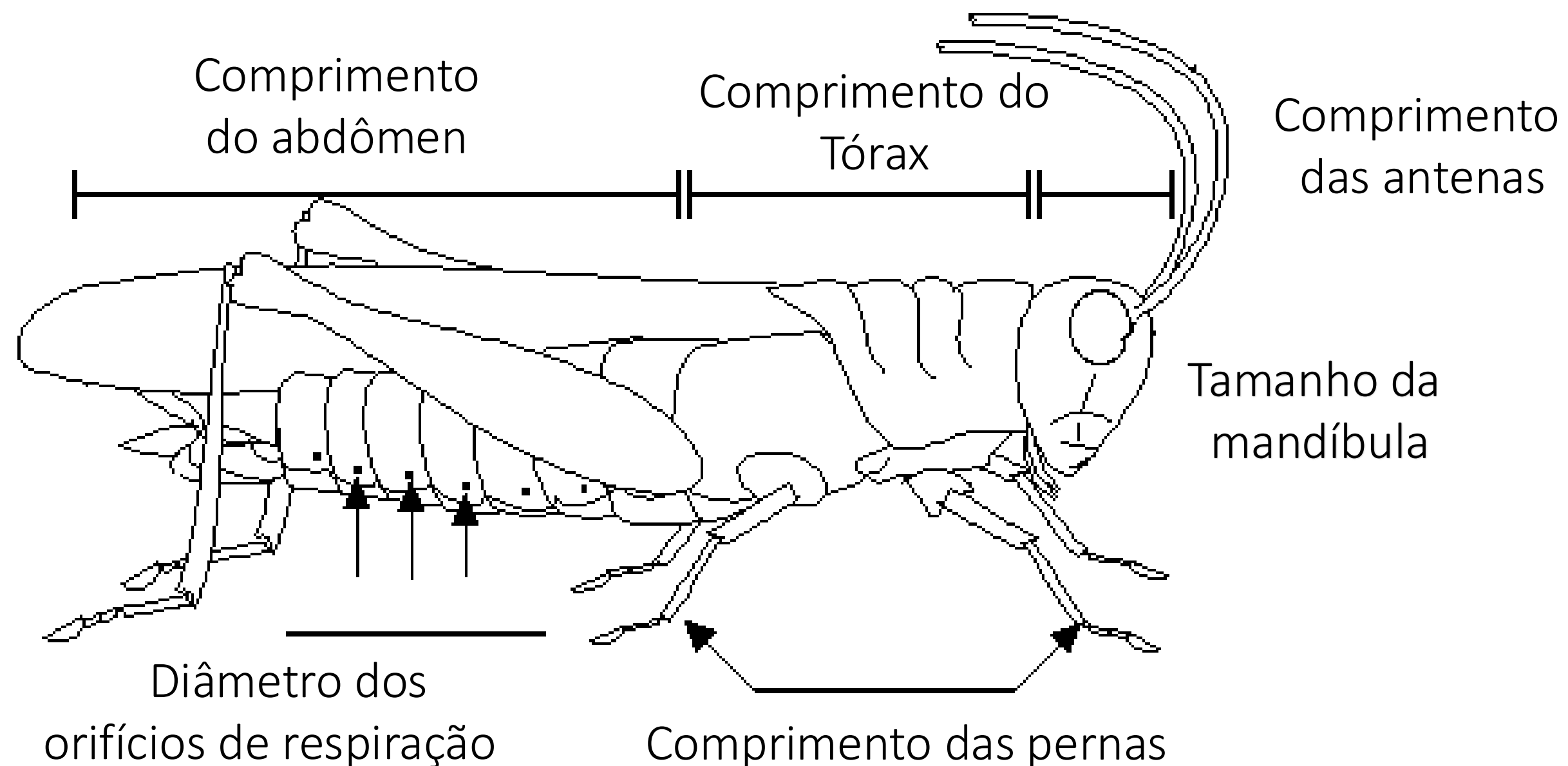
Gafanhoto



Para qualquer domínio de interesse podemos medir características

Cor: {Verde, Marrom, Cinza, Outra}

Tem asas?



Base de Treinamento

Podemos armazenar as características em *datasets*

O problema de classificação agora pode ser expresso da seguinte forma:

- Dada uma base de treino (Base), preveja o rótulo da classe dos exemplos ainda não vistos

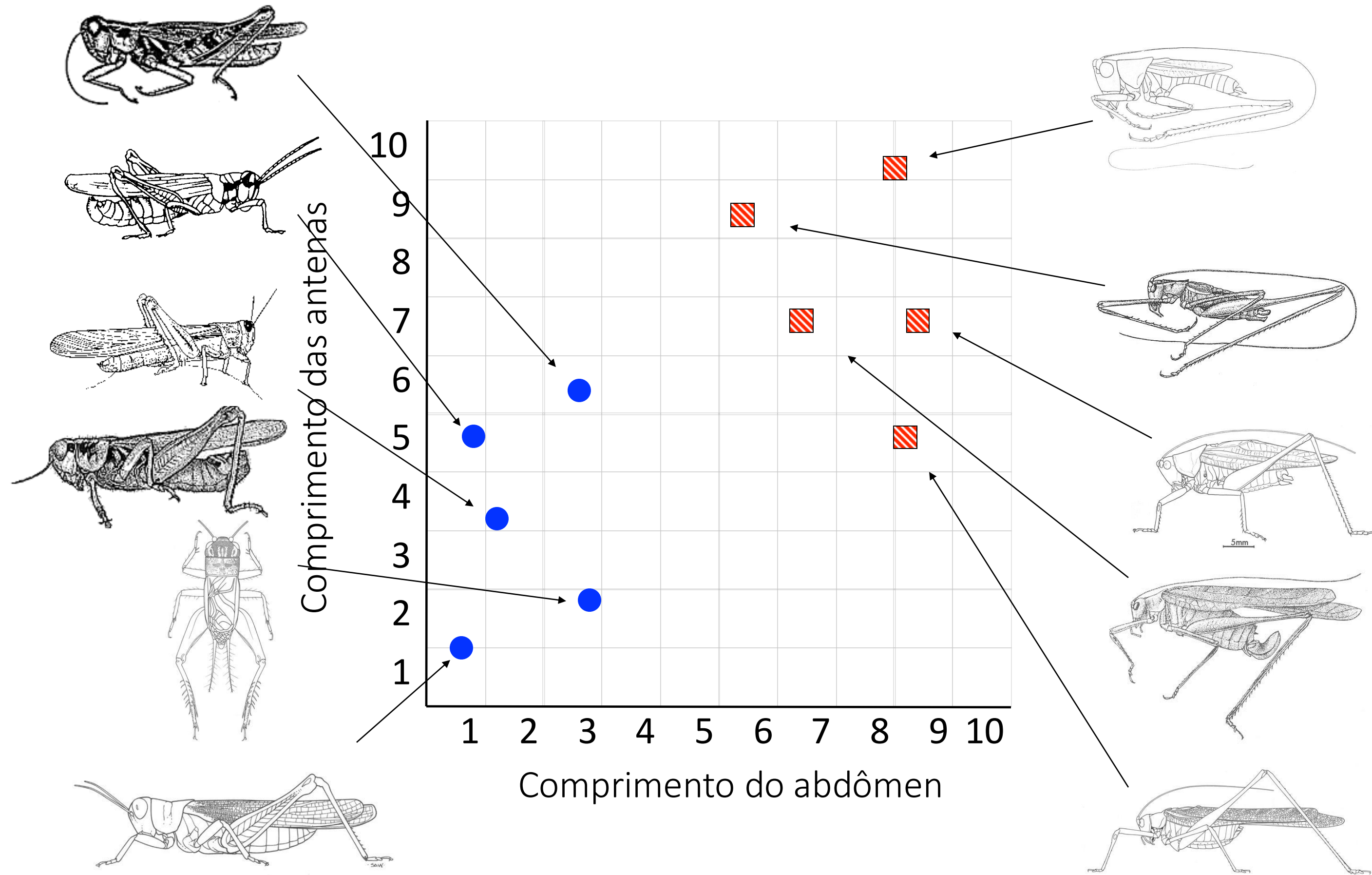
ID do inseto	Comp. do abdômen	Comp. das antenas	Classe do inseto
1	2.7	5.5	Gafanhoto
2	8.0	9.1	Esperança
3	0.9	4.7	Gafanhoto
4	1.1	3.1	Gafanhoto
5	5.4	8.5	Esperança
6	2.9	1.9	Gafanhoto
7	6.1	6.6	Esperança
8	0.5	1.0	Gafanhoto
9	8.3	6.6	Esperança
10	8.1	4.7	Esperança

Exemplo não visto =

11	5.1	7.0	????????????
----	-----	-----	--------------

Gafanhoto

Esperança



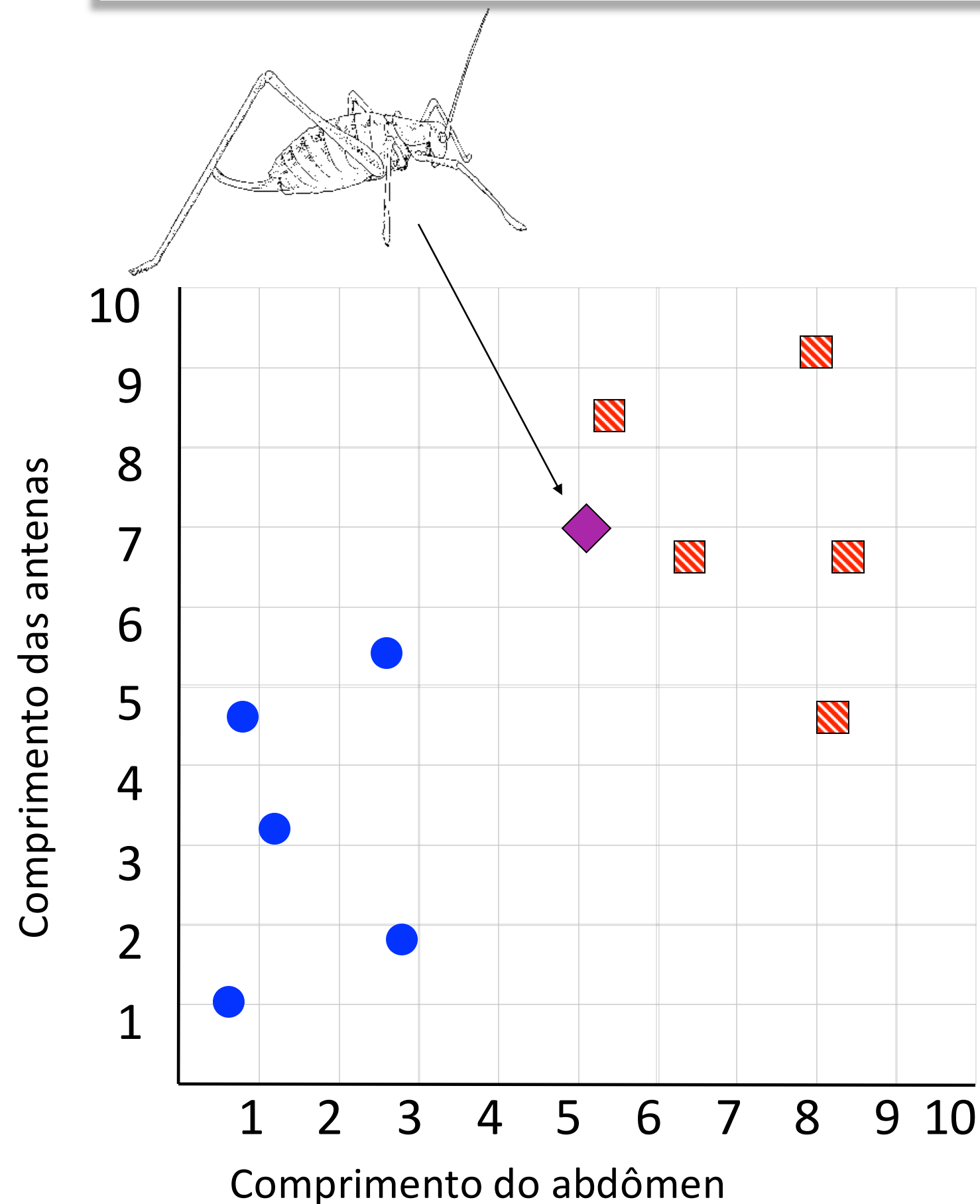
Exemplo não visto antes =

11

5.1

7.0

???????

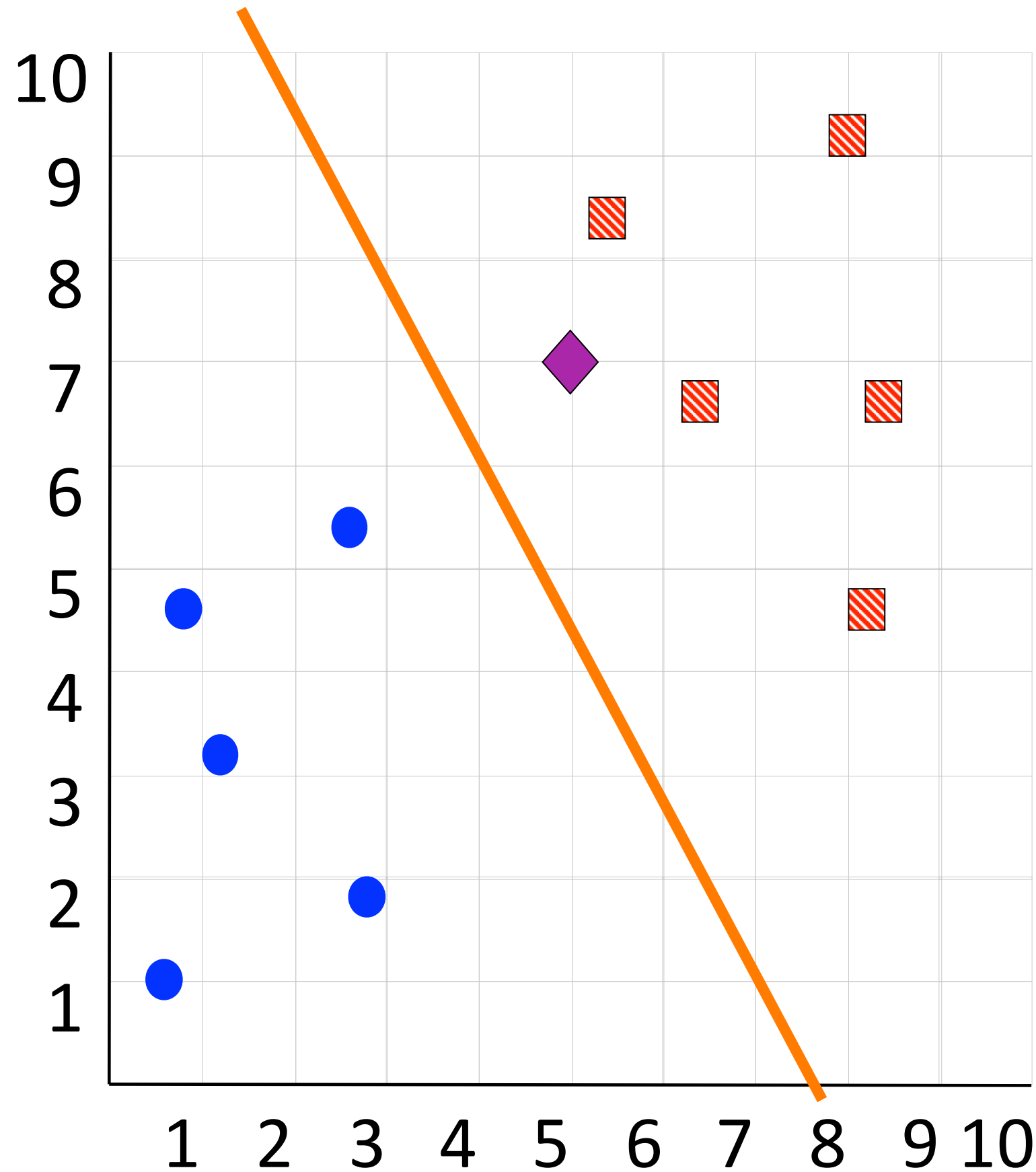


- Podemos **projetar** o exemplo não visto antes dentro do mesmo espaço que os dados de treino
- Acabamos de abstrair os detalhes do nosso problema particular. Será muito mais fácil falar de pontos no espaço

Esperança

Gafanhoto

Definindo classificação formalmente:



$$\mathbf{x}^{(i)} = \left[x_j^{(i)} \right]_{j=1}^m \in X^m$$

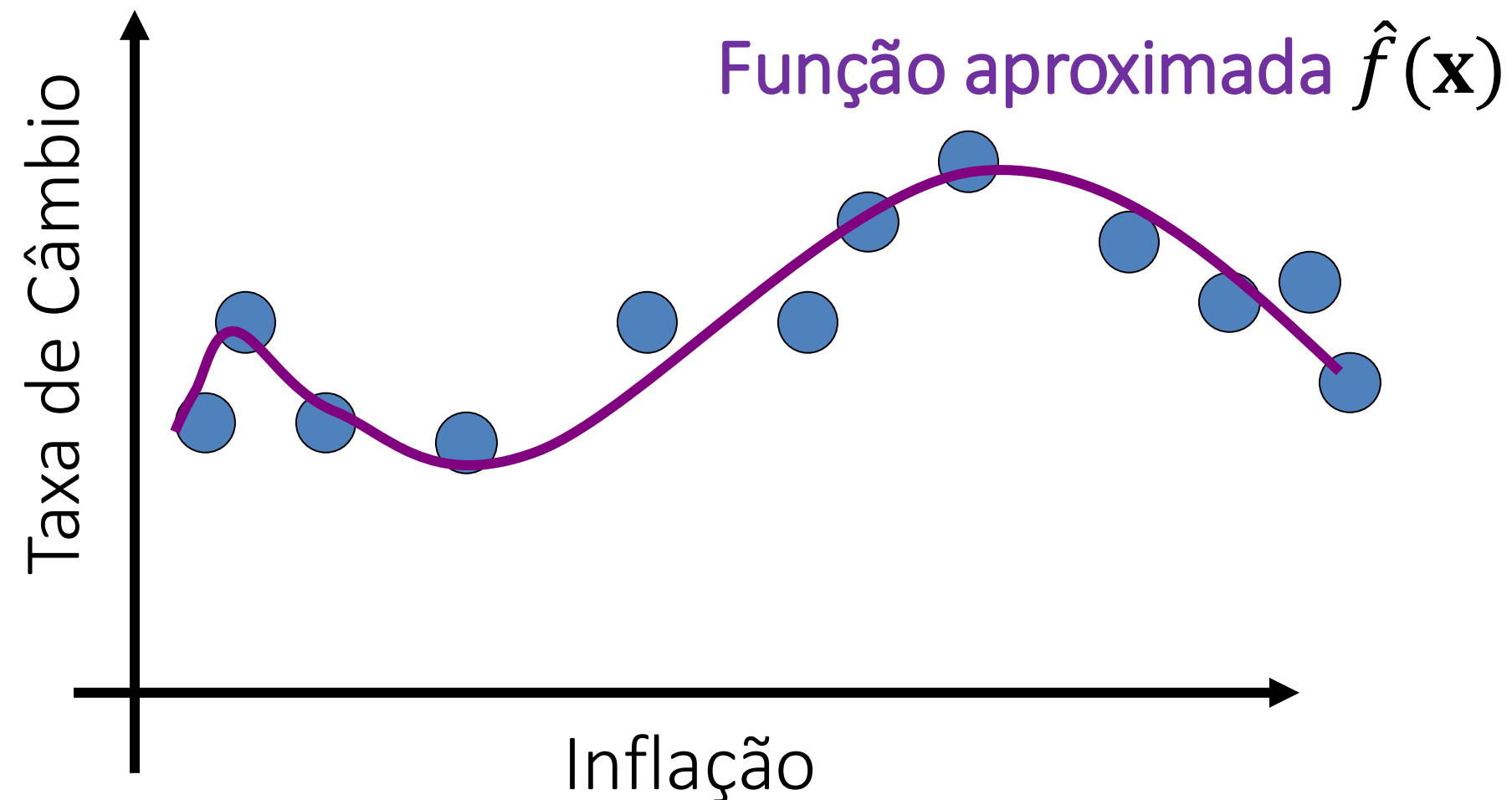
$$Y = \{y_1, \dots, y_k\}$$

$$D = \{ \mathbf{x}^{(i)}, f(\mathbf{x}^{(i)}) \}_{i=1}^N$$

$$\hat{f} = X^m \rightarrow Y$$

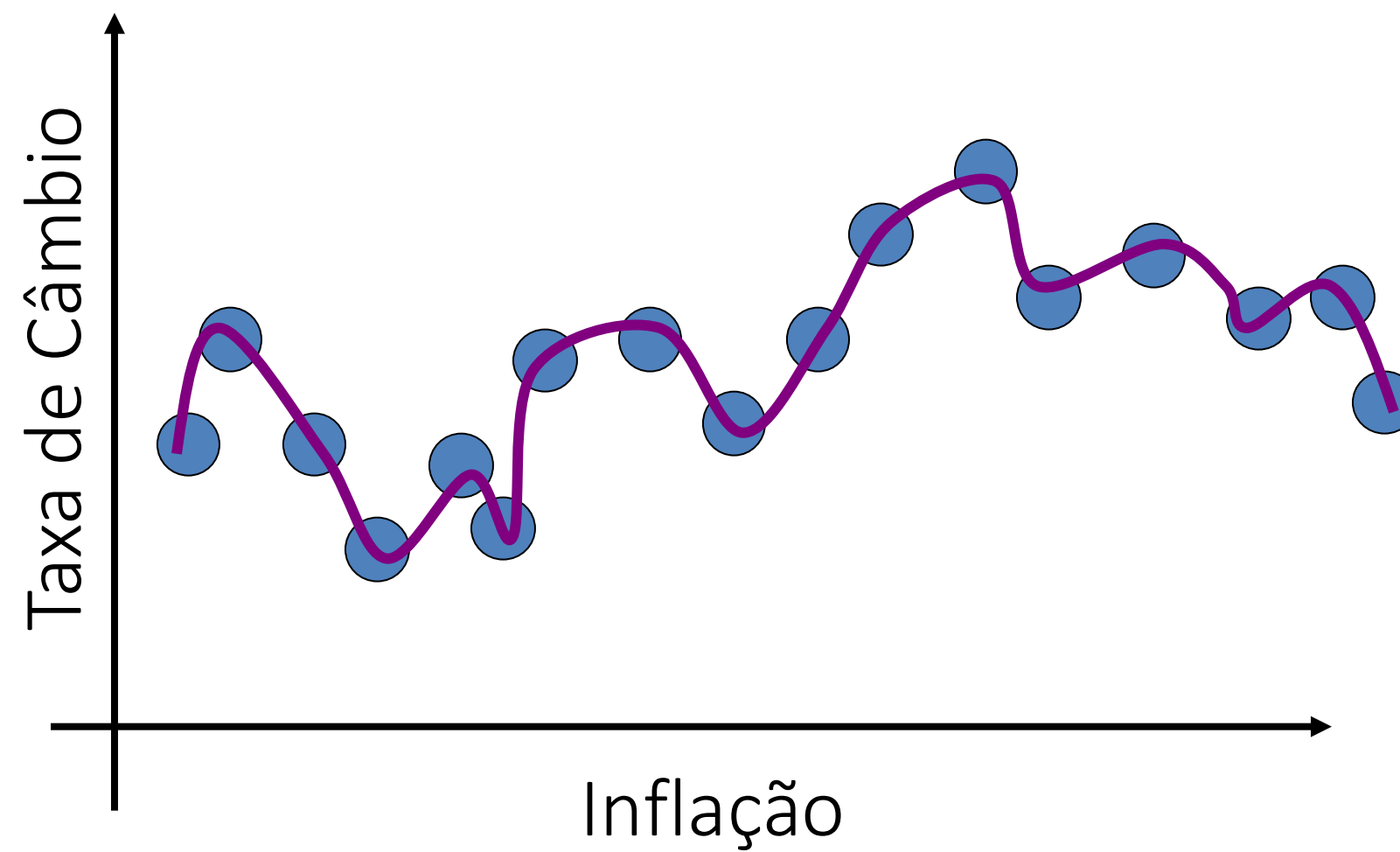
Problema de Regressão

- Funciona exatamente como a classificação, mas atributo meta é contínuo em vez de discreto
- Também pode ser visto sob a ótica de aproximação de funções
 - Descobrir a função que mapeia os atributos preditivos em um valor real
 - Em geral, busca-se minimizar uma função de custo
 - Erro quadrático médio, etc.



Problema de Regressão

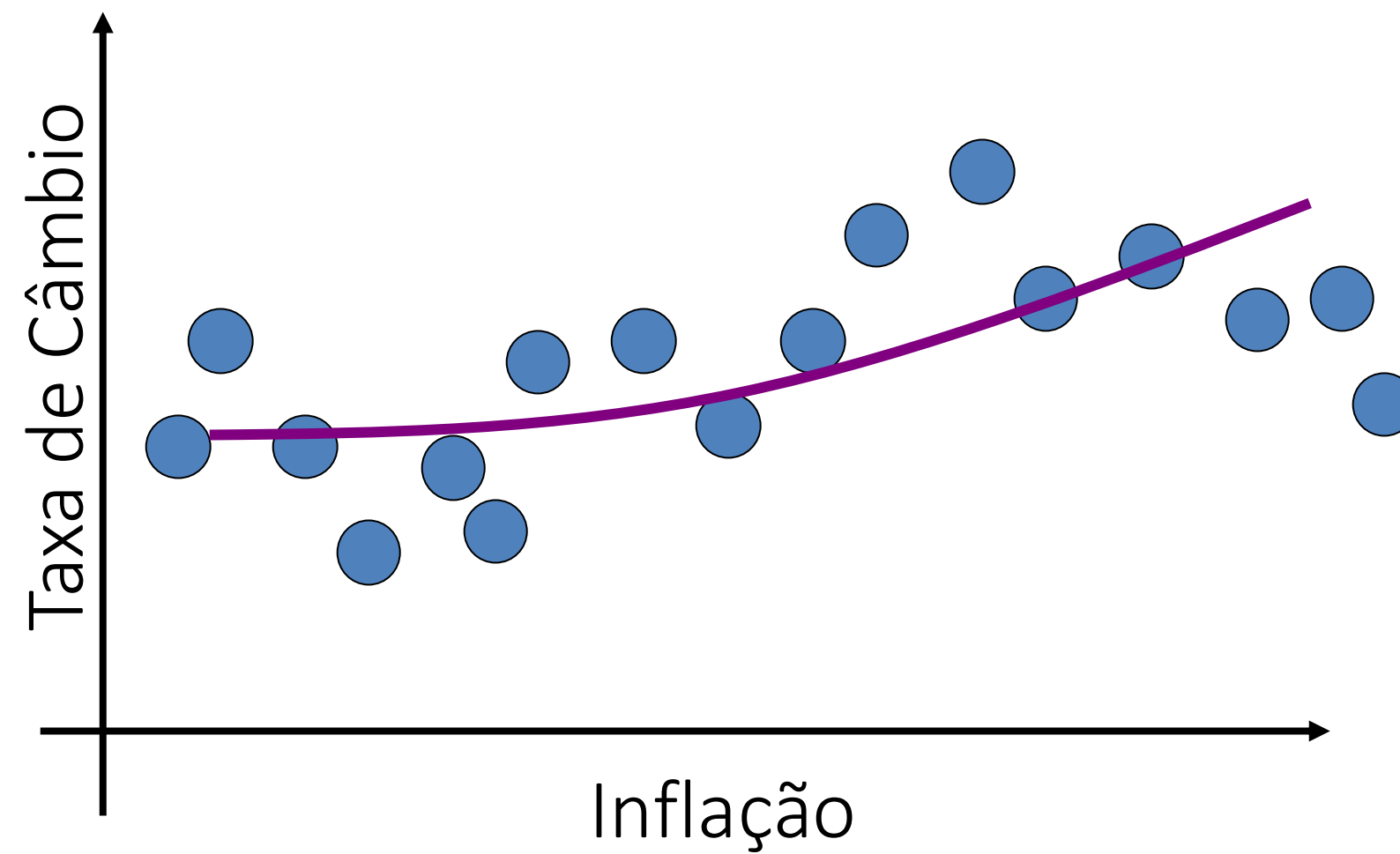
Overfitting



Também presente
em classificação

Problema de Regressão

Underfitting



Também presente
em classificação

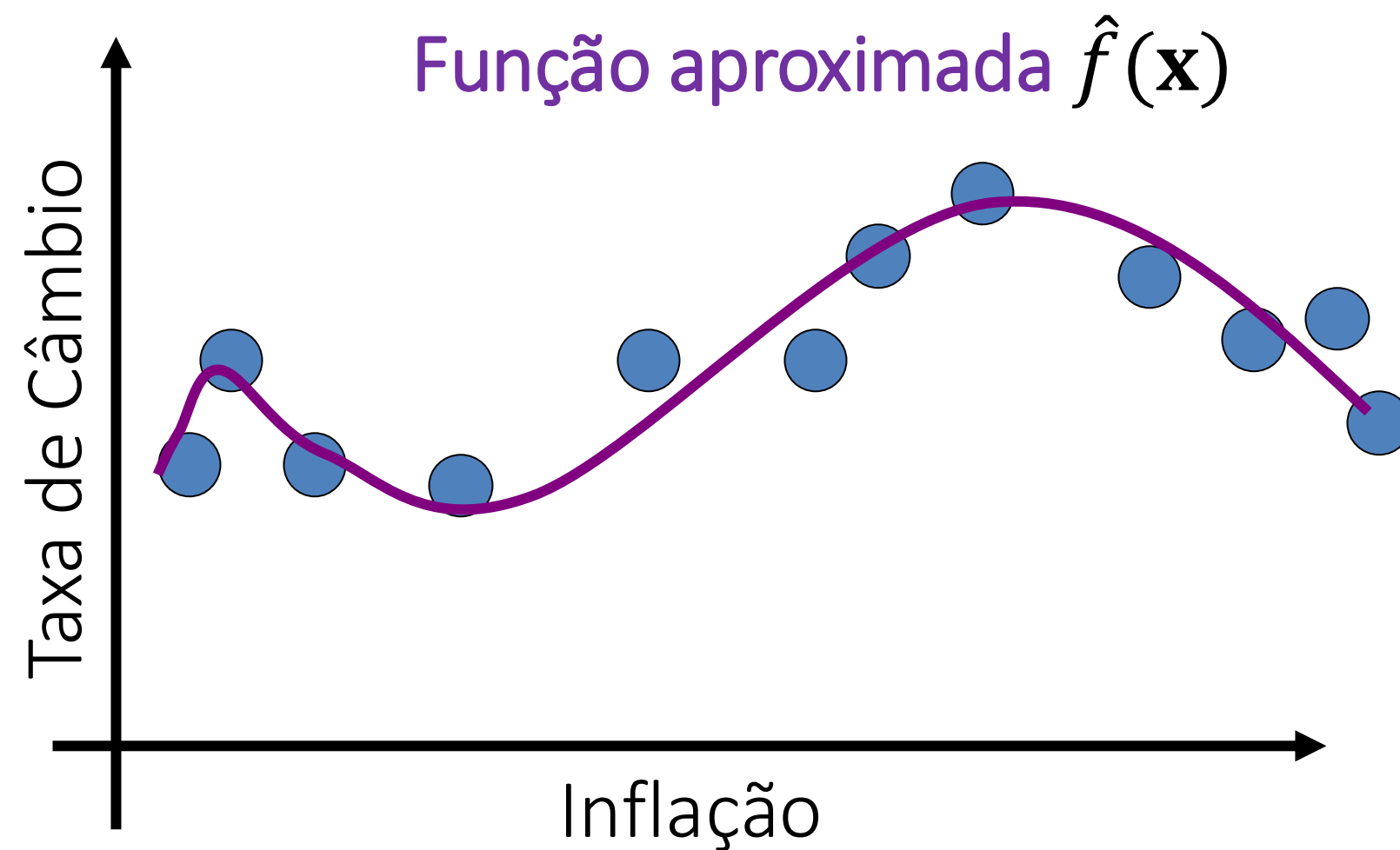
Formalizando Regressão

$$\mathbf{x}^{(i)} = \left[x_j^{(i)} \right]_{j=1}^m \in X^m$$

$$f(\mathbf{x}^{(i)}) \in \mathbb{R}$$

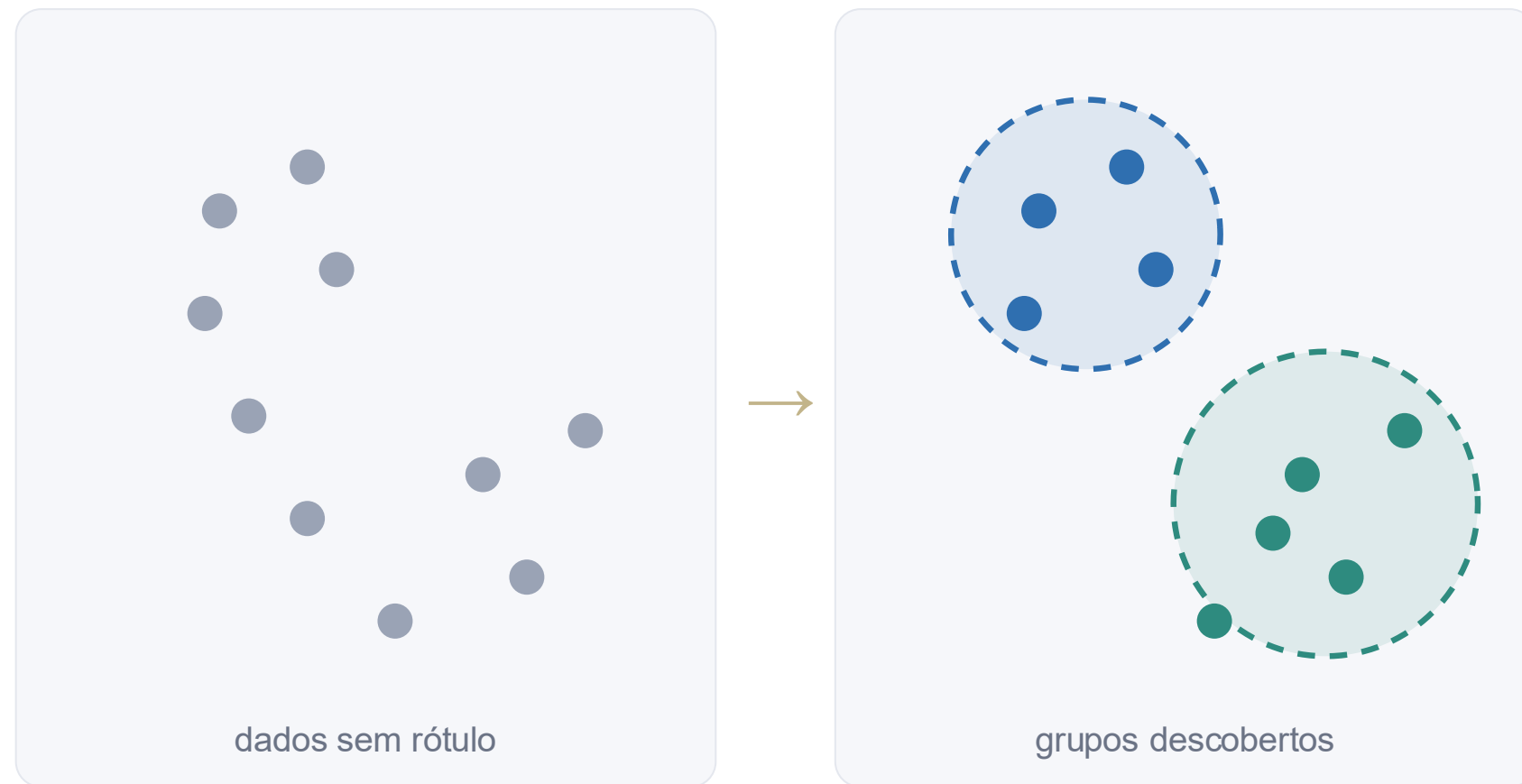
$$D = \{ \mathbf{x}^{(i)}, f(\mathbf{x}^{(i)}) \}_{i=1}^N$$

Descobrir $\hat{f}$ que aproxima f
minimizando uma função de erro e (ou $\mathcal{L}$)



Aprendizado não-supervisionado

Sem rótulos, o algoritmo descobre **sozinho** a estrutura escondida nos dados.



O algoritmo agrupa por **proximidade**, sem ninguém dizer as classes

■ Não há resposta certa: o objetivo é **revelar padrões**.

TAREFAS TÍPICAS

Agrupamento

Achar **grupos** naturais. Segmentar clientes.

Associação

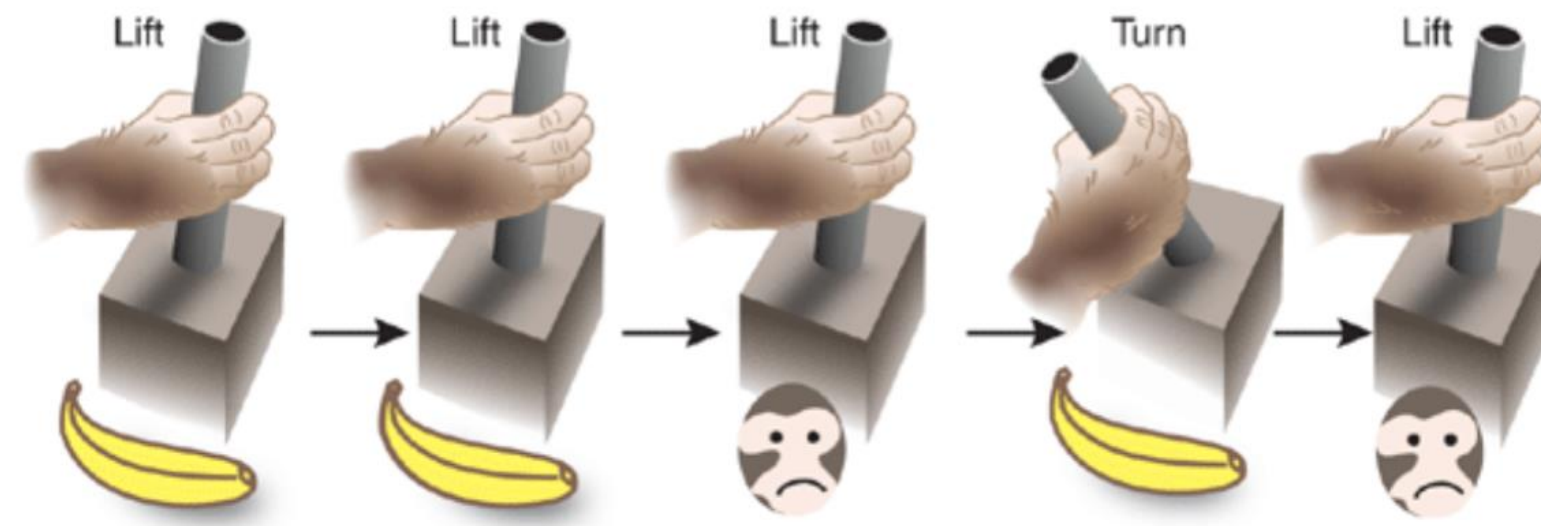
Achar **regras**. Cesta de compras.

Redução de dimensionalidade

Resumir variáveis em poucos eixos. PCA.

Aprendizado por Reforço

- Guiado por um crítico externo
 - Processo de tentativa e erro
 - Procura maximizar sinal de reforço
- Se ação tomada por sistema é seguida por estado satisfatório, o sistema é fortalecido. Caso contrário, o sistema é enfraquecido (Lei de Thorndike)
- Tipos de reforço
 - Positivo = recompensa
 - Negativo = punição
 - Nulo



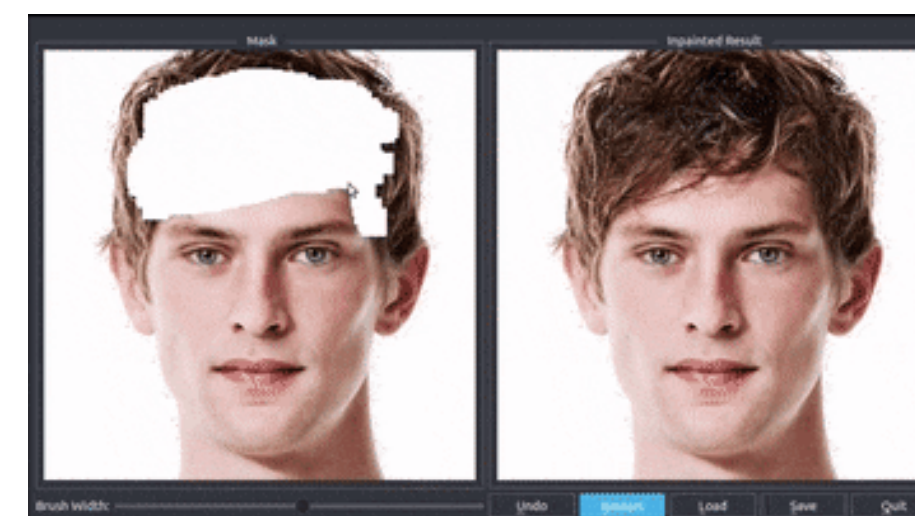
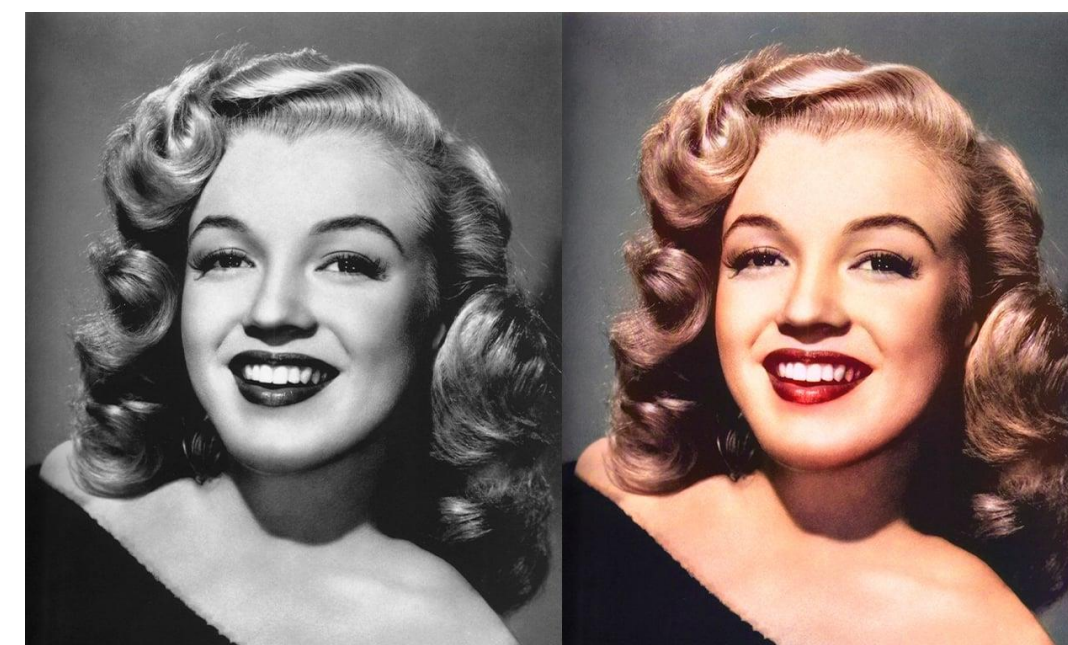
Aprendizado Auto-Supervisionado

- **Aprendizado supervisionado com rótulos “gratuitos”**

- Exemplos:

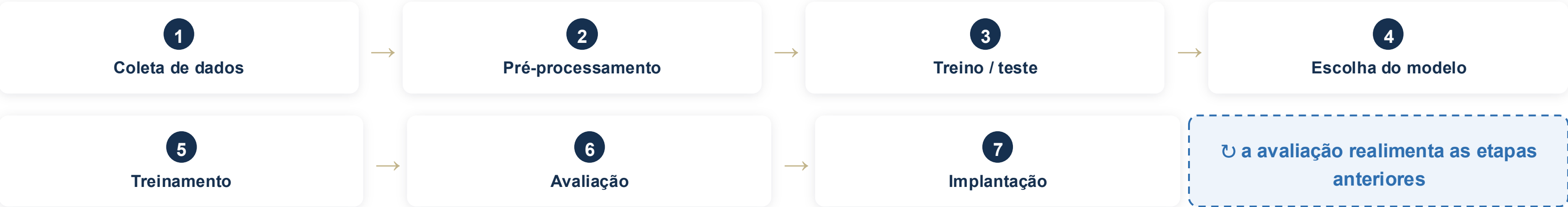
- Language modeling: geração de textos onde o rótulo (supervisão) é parte do texto já existente
- Colorir imagens: o rótulo é a cor original da imagem
- Reconstruir imagens: o rótulo é a parte da imagem que foi escondida/danificada durante o treinamento

The cat



O fluxo clássico de um projeto

Da coleta à produção, o trabalho segue etapas encadeadas, e quase sempre **iterativas**.



Modelo × algoritmo de aprendizagem

Algoritmo de aprendizagem

O **procedimento** que ajusta os parâmetros usando os dados de treino.

Ex.: CART (árvores), gradiente descendente, backpropagation.

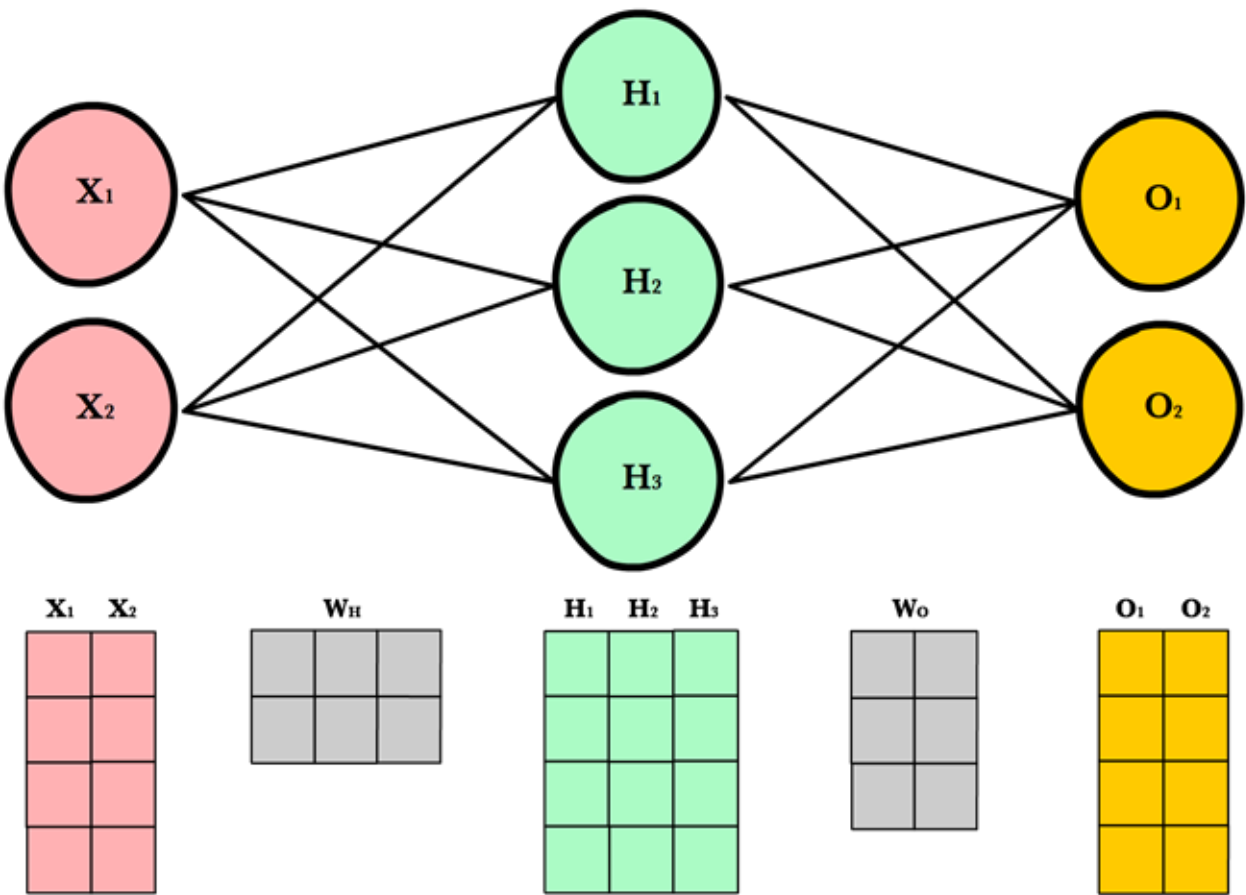
Algorithm 1 Backpropagation Algorithm

```
1: procedure TRAIN
2:    $X \leftarrow$  Training Data Set of size  $m \times n$ 
3:    $y \leftarrow$  Labels for records in  $X$ 
4:    $w \leftarrow$  The weights for respective layers
5:    $l \leftarrow$  The number of layers in the neural network,  $1 \dots L$ 
6:    $D_{ij}^{(l)} \leftarrow$  The error for all  $i, j$ 
7:    $t_{ij}^{(l)} \leftarrow 0$ . For all  $i, j$ 
8:   For  $i = 1$  to  $m$ 
9:      $a^l \leftarrow \text{feedforward}(x^{(i)}, w)$ 
10:     $d^l \leftarrow a(L) - y(i)$ 
11:     $t_{ij}^{(l)} \leftarrow t_{ij}^{(l)} + a_j^{(l)} \cdot t_i^{l+1}$ 
12:    if  $j \neq 0$  then
13:       $D_{ij}^{(l)} \leftarrow \frac{1}{m} t_{ij}^{(l)} + \lambda w_{ij}^{(l)}$ 
14:    else
15:       $D_{ij}^{(l)} \leftarrow \frac{1}{m} t_{ij}^{(l)}$ 
16:    where  $\frac{\partial}{\partial w_{ij}^{(l)}} J(w) = D_{ij}^{(l)}$ 
```

Modelo

O **resultado treinado** : a função aprendida que recebe entradas e devolve previsões.

Ex.: a árvore específica, os pesos da rede, os coeficientes da reta.



Onde ML entra no Data Mining

Data Mining

O **processo amplo** de descobrir conhecimento útil em dados: entender o problema, preparar, modelar, avaliar e implantar.

USA, NA
MODELAGEM



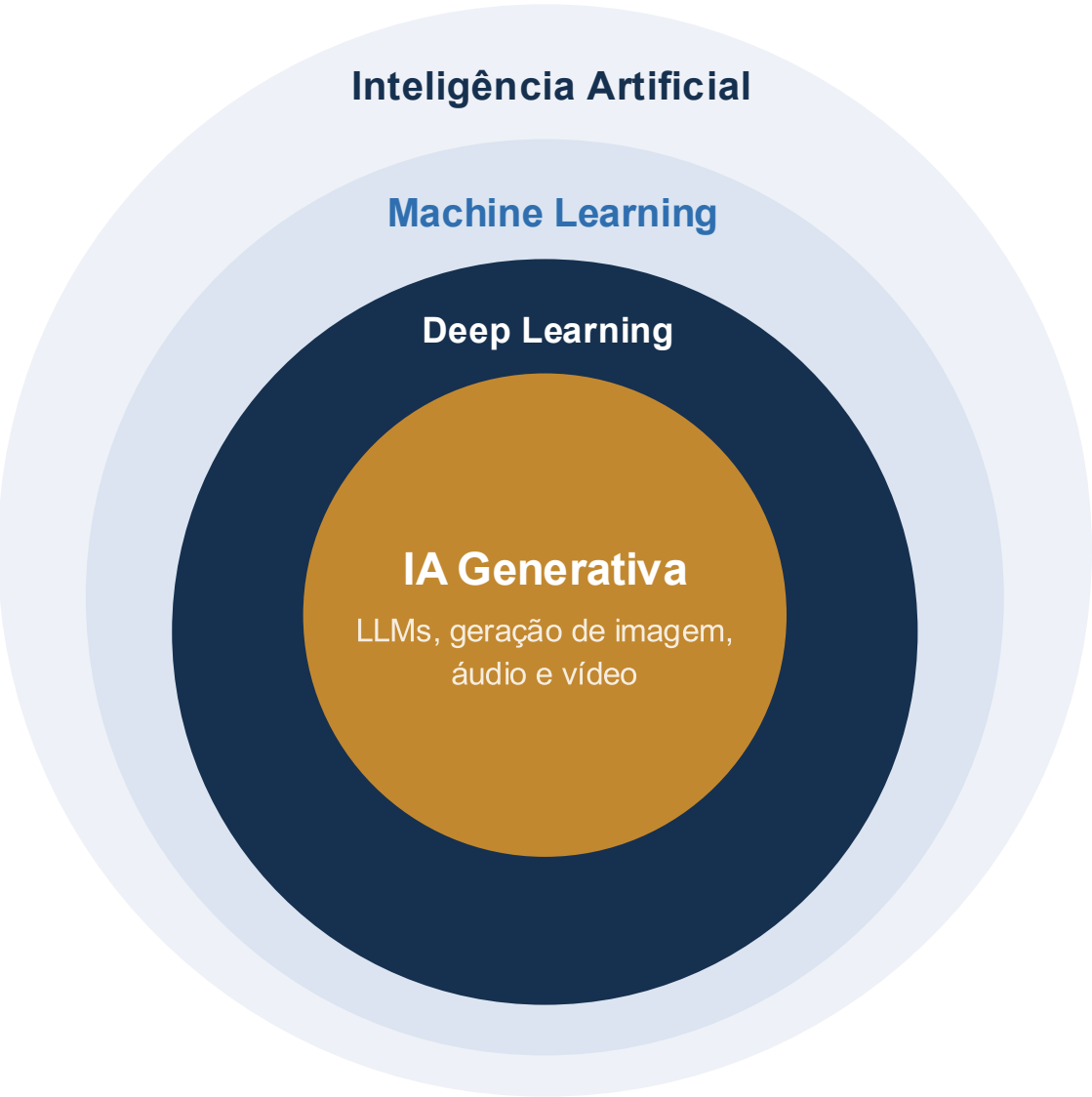
Machine Learning

O **conjunto de técnicas** que aprendem padrões a partir de dados. É a ferramenta da etapa de **modelagem**.

Nem todo Data Mining usa ML, e ML serve a muito além do Data Mining, mas é no encontro dos dois que está a maior parte do trabalho prático.

IA, Machine Learning e Deep Learning

Não são sinônimos nem concorrentes: são **camadas encaixadas**, cada uma um subconjunto da anterior.



Inteligência Artificial

O campo amplo: máquinas que exibem comportamento "inteligente" — **inclui regras escritas à mão**.

Machine Learning

Subconjunto da IA: **aprende padrões a partir de dados**, sem regras explícitas.

Deep Learning

Subconjunto do ML: **redes neurais profundas** que aprendem representações — imagem, áudio, texto.

IA Generativa

Está **dentro do Deep Learning**: redes profundas treinadas para **gerar conteúdo novo**.

MÓDULO 02

Aprendizado Supervisionado

Aprender uma função a partir de exemplos rotulados: k -NN, árvores, regressões, e o equilíbrio entre viés e variância.

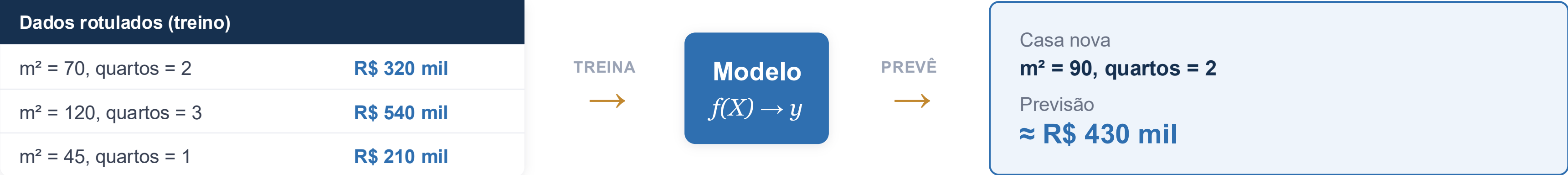
02

Aprender uma função a partir de exemplos

Damos ao modelo pares (entrada, resposta).

Ele aprende a função $f: X \rightarrow y$ que melhor explica os exemplos, e que **generaliza** para casos novos.

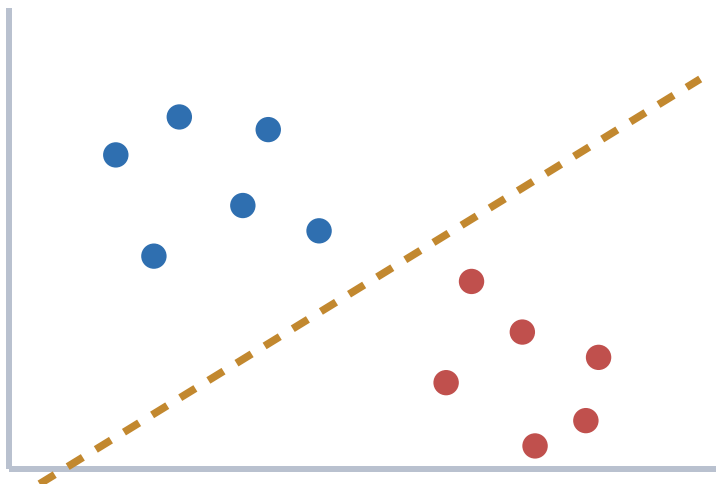
Cada modelo parte de um pressuposto diferente!



Classificação × regressão

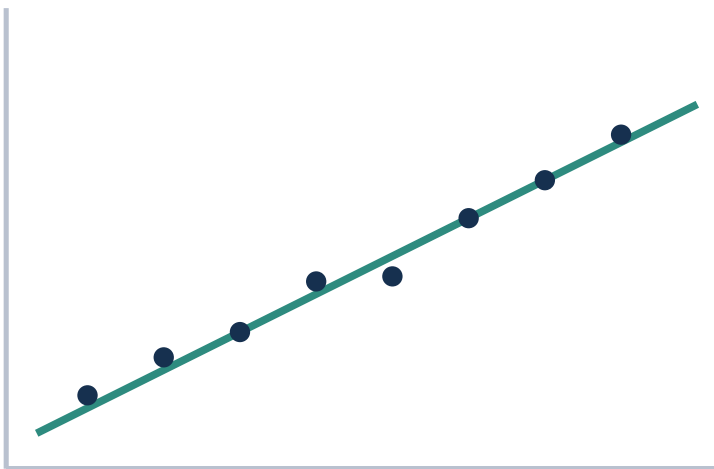
Classificação

Saída é uma **categoria**. O modelo traça uma **fronteira** entre as classes.



Regressão

Saída é um **valor contínuo**. O modelo ajusta uma **curva** aos pontos.



Alguns modelos de Aprendizado Supervisionado

Modelo	Como decide	Interpretável?	Quando preferir
k-NN	Pela maioria dos vizinhos mais próximos	Baixa (sem modelo explícito)	Fronteiras irregulares; poucos dados, bem normalizados
Árvore de decisão	Sequência de perguntas (cortes nos eixos)	Alta (regras legíveis)	Quando explicar a decisão importa; dados mistos
Regressão linear	Reta que minimiza o erro quadrático	Alta (coeficientes)	Prever um valor com relação ~linear
Regressão logística	Sigmoide → probabilidade → limiar	Alta (coeficientes)	Classificar com fronteira linear e probabilidades
Ensembles	Combinam muitos modelos por voto ou média	Média (caixa-preta)	Máxima precisão; dados tabulares complexos

O que é o k-NN?

Um modelo **supervisionado baseado em instâncias**: não constrói um modelo: guarda os exemplos e decide na hora, pela **vizinhança** do caso novo.

Supervisionado

Baseado em instâncias

Classificação ou regressão

COMO FUNCIONA

- 1 Mede a **distância** do caso novo a todos os exemplos guardados.
- 2 Seleciona os **k mais próximos** (os vizinhos).
- 3 **Vota** a classe da maioria, ou faz a **média** (regressão).

RESULTADO FINAL

Uma **classe** (ex.: categoria do produto) ou um **valor numérico** para cada novo caso, herdado dos vizinhos mais parecidos.

Como o k-NN funciona

1

Guardar

Armazena todos os exemplos rotulados — não há "treino".

2

Medir distância

Para um ponto novo, calcula a distância a todos os exemplos.

3

Selecionar k

Fica com os k exemplos mais próximos.

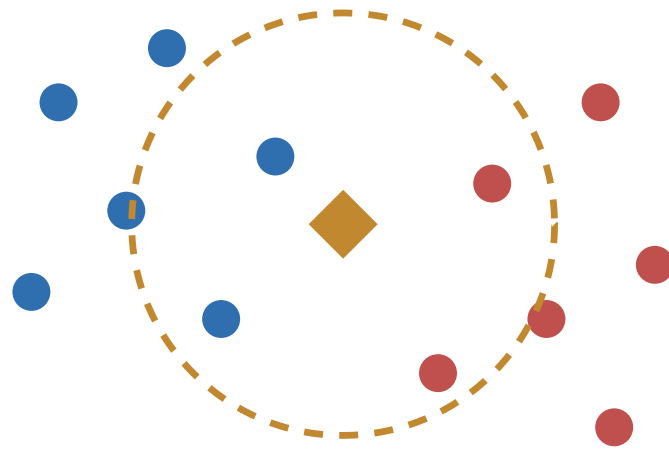
4

Votar

Classe = maioria dos vizinhos (ou média, em regressão).

Sem fase de treino: todo o custo está na **previsão**. Como tudo depende de distância, **normalizar os atributos é essencial**.

k-NN: a maioria dos vizinhos decide



- **Aprendizado baseado em instâncias** : guarda os exemplos, não constrói um modelo explícito.
- Para um ponto novo (◆), encontra os **k mais próximos** e adota a **classe da maioria** .

Com **k = 5** : 3 azuis e 2 vermelhos dentro do círculo → o ponto novo é classificado como **azul**.

O papel de k e da distância

A distância

Mede quão parecidos são dois pontos. A mais comum é a **euclidiana**:

$$d(p, q) = \sqrt{\sum_i (p_i - q_i)^2}$$

Atributos em escalas diferentes distorcem a distância → **normalizar é essencial**.

O valor de k

- **k pequeno** (ex.: 1): fronteira **irregular**, sensível a ruído → overfitting.
- **k grande**: fronteira **suave**, mas pode ignorar detalhes → underfitting.

Escolhe-se k por **validação**; ímpar evita empates em duas classes.

Classificar um produto novo no catálogo

Um produto novo entra entre os **+22 mil itens** sem categoria. O k-NN acha os produtos mais parecidos (preço, peso, material) e adota a **categoria da maioria** .

Produto novo

Lâmina inox 18 cm

preço R\$ 34 peso 120 g cabo polímero

K = 5
→

Vizinho mais próximo	Distância	Categoria
Faca chef 20 cm	0,12	Cutelaria
Faca legumes	0,15	Cutelaria
Espátula	0,19	Cozinha
Faca desossa	0,21	Cutelaria
Concha	0,24	Cozinha

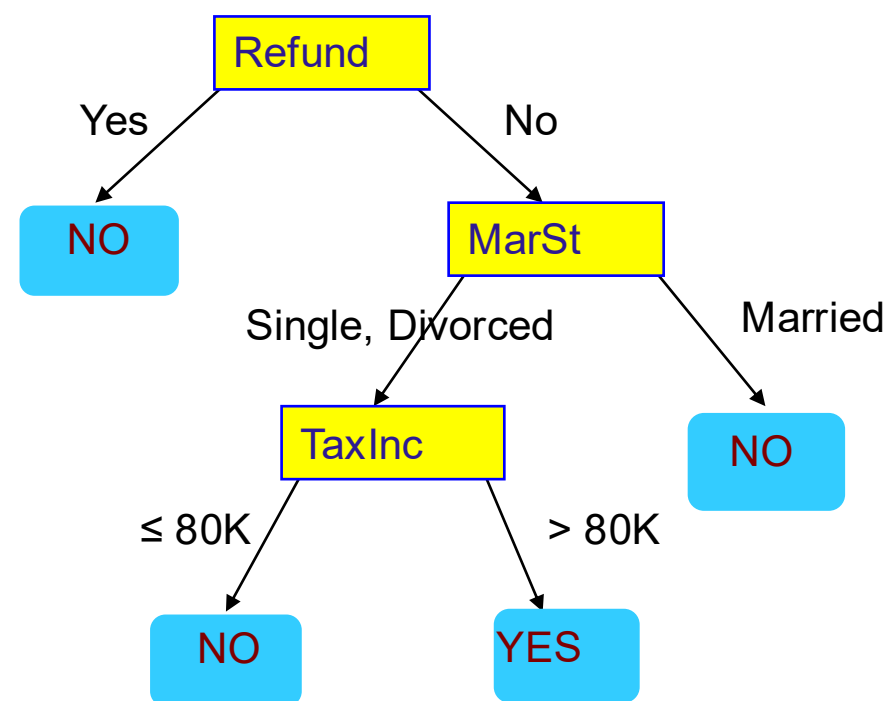
RESULTADO

3 Cutelaria × 2 Cozinha → categoria prevista: Cozinha

Atributos em escalas diferentes (R\$ × g) → normalizar é essencial antes de medir distância.

O que é uma árvore de decisão?

Um modelo **supervisionado** que faz uma **sequência de perguntas** sobre os atributos, organizadas numa hierarquia de nós e folhas.



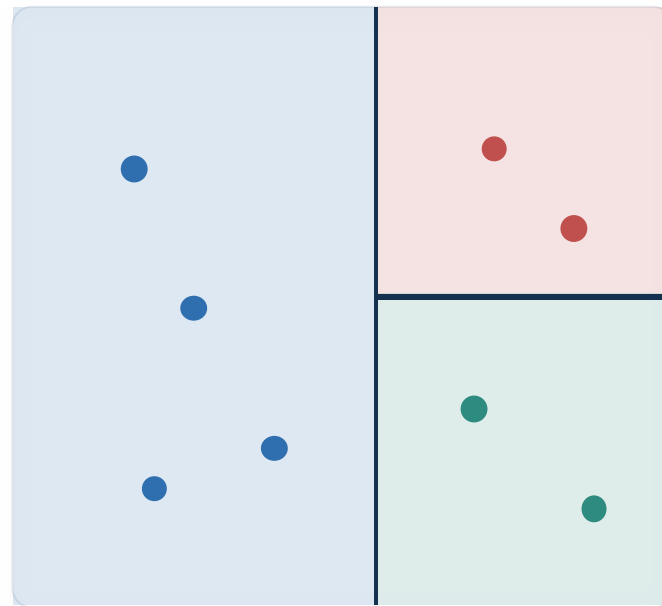
COMO FUNCIONA

- 1 Escolhe a pergunta que **melhor separa** as classes (Gini / entropia).
- 2 **Ramifica** e repete o processo em cada ramo.
- 3 Cada **folha** é uma decisão final.

RESULTADO FINAL

Um conjunto de **regras legíveis** (se... então...) → uma classe ou valor. O caminho até a folha é a **explicação**.

Como divide o espaço



Fronteiras retangulares (cortes nos eixos)

- Cada corte é **paralelo a um eixo** (um teste de cada vez) → regiões retangulares.
- Lida bem com atributos **numéricos e categóricos**, sem normalizar.

Cuidado: árvores muito profundas **decoram** o treino. Poda e profundidade máxima controlam o overfitting.

Como a árvore é construída

1

Começar na raiz

Todos os exemplos juntos em um só nó.

2

Melhor pergunta

Escolhe o corte que mais separa as classes (Gini / ganho).

3

Dividir

Quebra os dados em ramos conforme a resposta.

4

Repetir

Aplica em cada nó até ficar puro ou atingir um limite.

5

Podar

Corta ramos que não generalizam (anti-overfitting).

É um algoritmo **guloso**: melhor escolha local em cada nó. O resultado são **regras if-then legíveis**.

O que são as regressões?

Regressão linear

Ajusta uma **reta** aos dados. Serve para **prever um número**.

RESULTADO FINAL

Um valor contínuo

ex.: preço de tabela, peso, tempo

Regressão logística

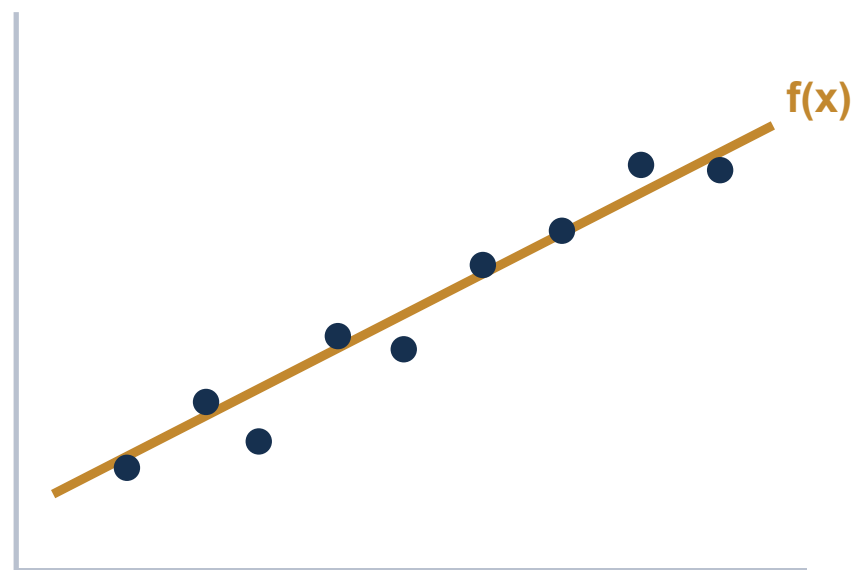
Apesar do nome, **classifica**: sigmoide → **probabilidade** → classe.

RESULTADO FINAL

Uma probabilidade (0–1) → classe

ex.: é Cutelaria? sim/não

Ajustar a melhor reta



$$\hat{y} = \beta_0 + \beta_1 \cdot x$$

- β_0 : intercepto · β_1 : inclinação (quanto y muda por unidade de x).
- Ajusta os coeficientes **minimizando o erro quadrático** (mínimos quadrados).

Simples e **interpretável**; supõe relação aproximadamente linear.

Como a regressão linear funciona

1

Supor uma reta

$y = b_0 + b_1 \cdot x_1 + \dots$: relação linear entre entradas e saída.

2

Medir o erro

Soma dos erros quadráticos entre previsto e real.

3

Ajustar coeficientes

Encontra os b que minimizam o erro (mínimos quadrados).

4

Prever

Aplica a reta ajustada a novos dados.

Os **coeficientes são interpretáveis** (o efeito de cada atributo), mas o modelo é **sensível a outliers**.

Estimar o preço de tabela do produto novo

O **mesmo produto novo** entra no catálogo. Em vez da categoria, a regressão prevê um **número**, o preço sugerido, a partir dos mesmos atributos.

$$\text{preço} \approx 6 + 0,10 \cdot \text{peso(g)} + 0,50 \cdot \text{lâmina(cm)} + 7 \cdot (\text{é aço inox})$$

- Cada **+100 g** de peso → **+R\$ 10** no preço.
- Ser **aço inox** soma **+R\$ 7** ao valor de tabela.

Lâmina inox 18 cm · 120 g

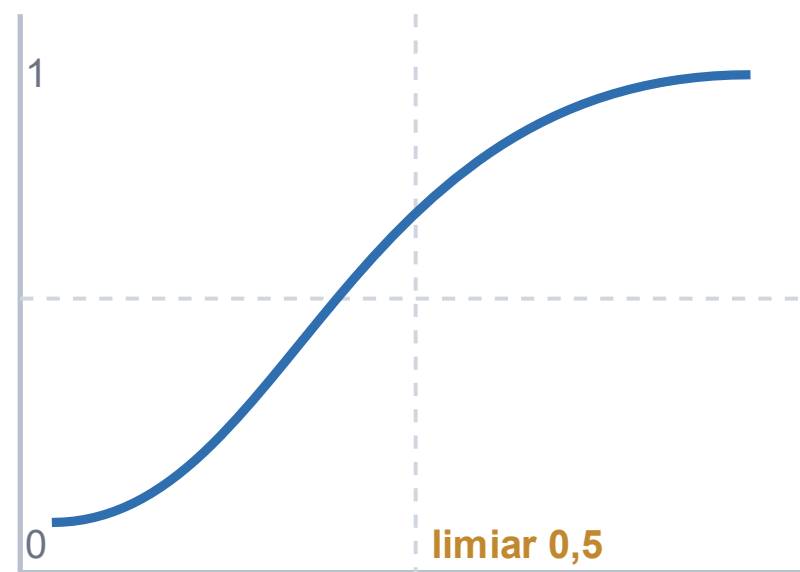
$$6 + 0,10 \cdot 120 + 0,50 \cdot 18 + 7$$

RESULTADO FINAL · UM NÚMERO

≈ R\$ 34 preço de tabela

Alimenta a **precificação** e o **cadastro** do produto.

De reta a probabilidade



$$p = \sigma(\beta_0 + \beta_1 x) = 1 / (1 + e^{-z})$$

- A **sigmoide** comprime qualquer valor em uma **probabilidade** (0 a 1).
- Um **limiar** (ex.: 0,5) transforma a probabilidade em **classe**.

Fronteira de decisão **linear**; coeficientes ainda interpretáveis.

Como a regressão logística funciona

1

Combinar linearmente

Soma ponderada dos atributos, como na linear.

2

Passar pela sigmoide

Comprime o resultado em uma probabilidade de 0 a 1.

3

Ajustar pesos

Maximiza a verossimilhança dos rótulos (gradiente).

4

Decidir com limiar

Prob. $\geq 0,5$ (ou outro corte) $\rightarrow$ classe positiva.

Apesar do nome, é **classificação**: entrega uma **probabilidade** ajustável pelo limiar e coeficientes interpretáveis.

Qual a probabilidade de ser Mais Vendidos?

O mesmo produto novo. A logística devolve a **probabilidade** de o item pertencer à categoria **Cutelaria** ; o limiar 0,5 decide a classe, agora com confiança.

Atributos do produto novo	
Peso	120 g
Comprimento	18 cm
Material do corpo	Aço inox
Cabo	Polímero

SIGMOIDE
→

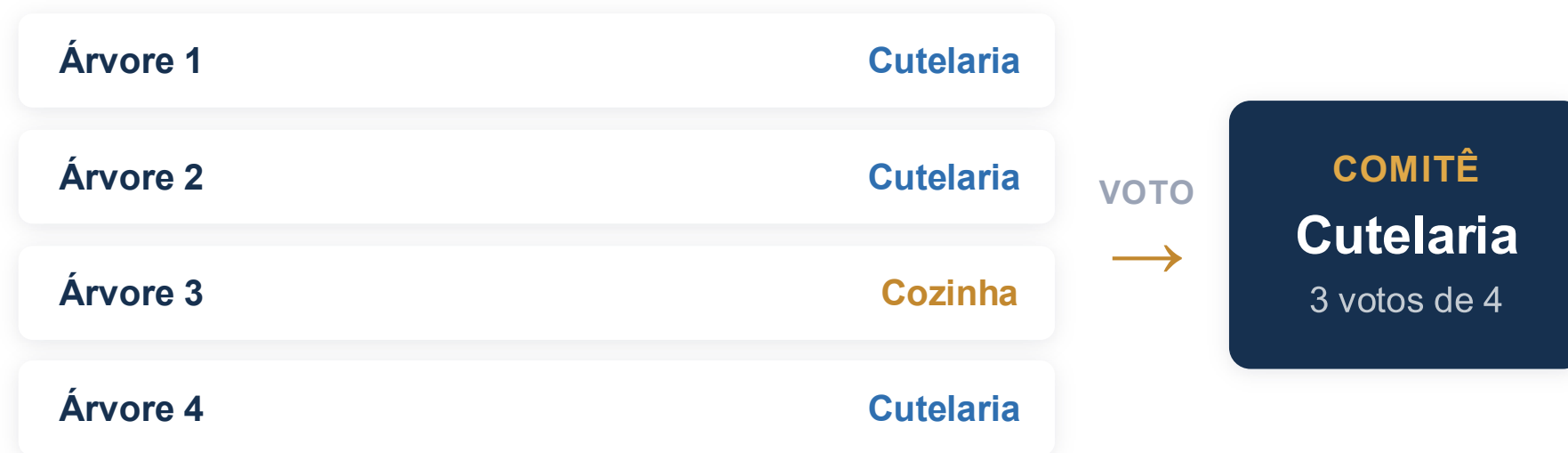
P(Mais Vendidos)

0,86

limiar = 0,50

RESULTADO 0,86 > 0,50 → classe **Mais Vendidos** , com alta confiança

Modelos de comitê (ensembles)



Por que funciona

- Cada modelo **erra de um jeito diferente**; juntos, os erros tendem a se **cancelar**.
- Exige modelos **diversos e melhores que o acaso** — a "sabedoria das multidões".
- Classificação → **voto** da maioria · regressão → **média** das previsões.

Na prática, **Random Forest** e **XGBoost** são campeões em dados tabulares e competições.

Duas formas de montar o comitê

Bagging

treina em paralelo

modelo

modelo

modelo



voto

- Cada modelo vê uma **amostra diferente** dos dados; combina por voto/média.

Reduz a **variância** (combate overfitting).

Ex.: **Random Forest** — floresta de árvores

Boosting

treina em sequência

modelo 1



modelo 2



modelo 3

- Cada novo modelo **corrige os erros** do anterior.

Reduz o **viés** (maior precisão).

Ex.: **Gradient Boosting**, XGBoost, AdaBoost

Lembrando o compromisso viés–variância: **bagging** ataca a **variância**, **boosting** ataca o **viés**.

Os modelos supervisionados, lado a lado

Modelo	Como decide	Interpretável?	Quando preferir
k-NN	Pela maioria dos vizinhos mais próximos	Médio (sem modelo explícito)	Fronteiras irregulares; poucos dados, bem normalizados
Árvore de decisão	Sequência de perguntas (cortes nos eixos)	Alta (regras legíveis)	Quando explicar a decisão importa; dados mistos
Regressão linear	Reta que minimiza o erro quadrático	Alta (coeficientes)	Prever um valor com relação ~linear
Regressão logística	Sigmoide → probabilidade → limiar	Alta (coeficientes)	Classificar com fronteira linear e probabilidades
Ensembles	Combinam muitos modelos por voto ou média	Baixa (caixa-preta)	Máxima precisão; dados tabulares complexos

MÓDULO 03

Avaliação e Validação

Por que separar treino e teste, validação cruzada, a matriz de confusão e as métricas que dizem se o modelo é bom de verdade.

03

Uma prova é como avaliar um modelo

Imagine que você vai fazer uma **prova** e estuda resolvendo a **lista de exercícios**. Dois cenários:

Prova = lista de exercícios

Você **gabarita**... mas isso prova que aprendeu? Não: pode só ter **decorado** as respostas.

↳ é avaliar no treino

Prova com questões novas

Mesmo assunto, problemas **inéditos**. Ir bem aqui mostra que você **aprendeu de verdade**.

↳ é avaliar no teste

Avaliar no treino

99%

Acurácia inflada: o modelo já **viu** essas respostas. Não diz nada sobre o futuro.

vs

Avaliar no teste

82%

Dados **nunca vistos**: estima o desempenho real. É o número honesto.

Treino · validação · teste



Treino

Ajusta os **parâmetros** do modelo.

Validação

Ajusta **hiperparâmetros** e compara modelos.

Teste

Uma **única vez** , no fim: a estimativa final e imparcial.

Validação cruzada (k-fold)

Divide os dados em **k partes**; treina k vezes, cada vez deixando **uma parte de fora** para testar. O resultado é a **média** das k rodadas.



- Todos os dados servem de **treino e teste**, em rodadas diferentes.
- Reduz a dependência de uma **única divisão** de sorte (ou azar).

Estimativa mais **estável** e confiável, ao custo de treinar k vezes.

Matriz de confusão

	REAL	
	Prev: Pos	Prev: Neg
Pos	VP 45	FN 5
Neg	FP 8	VN 42

- **VP / VN:** o modelo acertou (positivo e negativo).
- **FP:** previu positivo, era negativo (alarme falso).
- **FN:** previu negativo, era positivo (deixou passar).

Todas as métricas de classificação saem **destes quatro números**.

Os erros tem pesos diferentes

CENÁRIO BANCÁRIO

Antifraude de cartão

FP · bloquear uma compra legítima

Falso bloqueio: cliente constrangido, atrito e risco de troca de banco. **Erro caro.**

FN · deixar passar uma fraude

Prejuízo pontual, em geral limitado ou coberto. **Mais tolerável.**

Prefere tolerar **FN** → limiar mais **permissivo**.

CENÁRIO DE SAÚDE

Triagem de uma doença

FN · não detectar quem está doente

Atraso no tratamento, agravamento, risco de vida. **Erro caro.**

FP · falso alarme em quem é saudável

Leva só a um exame extra; reversível. **Mais tolerável.**

Prefere tolerar **FP** → limiar mais **sensível**.

Não existe erro "grátis". O **limiar de decisão** apenas troca um erro pelo outro, escolher qual tolerar é uma decisão **de negócio**, guiada pelo custo de cada erro.

Acurácia, precisão, recall e F1

Acurácia

$$(VP+VN) / total$$

Fração de **acertos** sobre tudo.

Precisão

$$VP / (VP+FP)$$

Dos previstos positivos, quantos **eram mesmo**.

Recall

$$VP / (VP+FN)$$

Dos positivos reais, quantos foram **pegos**.

F1-score

$$2 \cdot P \cdot R / (P+R)$$

Média harmônica de precisão e recall.

Prev: defeito

Prev: ok

VP

92

FN

8

FP

150

VN

9750

98,4%

acurácia (engana)

92%

recall

38%

precisão

FN = defeito que vaza ao cliente: troca, garantia, marca. → priorizar **recall**.

FP = peça boa reprovada: desperdício e reinspeção. → o **F1** equilibra os dois.

Métricas de regressão: MAE, RMSE, R²

As três medem o erro de prever **números**. Para sentir a diferença, veja os mesmos erros em cada uma.

MAE

$$\text{média de } |y - \hat{y}|$$

Erro médio **absoluto**, na unidade original. Fácil de ler e de comunicar.

EXEMPLO · ERROS 2, 3, 10

$$\text{MAE} = 5$$

cada erro pesa igual

RMSE

$$\sqrt{\text{média de } (y - \hat{y})^2}$$

Eleva ao quadrado: penaliza mais os **erros grandes**. Sensível a outliers.

MESMOS ERROS 2, 3, 10

$$\text{RMSE} = 6,1$$

o erro 10 puxa para cima

R²

$$1 - \text{SQres} / \text{SQtot}$$

Fração da **variância explicada** pelo modelo, sem unidade.

0

= chutar a média

1

= perfeito

MAE × RMSE: se errar muito de vez em quando é grave, prefira **RMSE**; se todo erro pesa igual, **MAE**. O **R²** diz o quanto o modelo supera o simples chute da média.

Manutenção preditiva: dias até a falha

O modelo prevê **quantos dias faltam até a próxima falha** de uma prensa de estampagem. Três números orientam a manutenção:

MAE

4 dias

Erro **típico** da previsão. Fácil de comunicar ao time de manutenção.

Leitura: "em média erramos **~4 dias** para mais ou para menos".

RMSE

9 dias

Mais que o dobro do MAE: há previsões **muito erradas**, e são as caras.

RMSE >> MAE → existem **erros grandes** escondidos na média.

R²

0,88

Explica **88%** da variação da vida útil das prensas.

Muito acima de um **cronograma fixo** de troca por tempo.

A discussão: prever a falha **tarde demais** → parada não planejada da linha e perda de produção (erro caro); prever **cedo demais** → troca de peça e manutenção antecipada (desperdício menor). Como os erros grandes são os mais caros, o **RMSE** é a métrica que mais pesa aqui.

Comparar modelos e escolher

Modelo	Acurácia	F1	AUC
k-NN	0,84	0,81	0,88
Árvore	0,82	0,80	0,85
Reg. logística ★	0,89	0,87	0,93

- Compare a métrica que **importa para o problema** , na validação cruzada.
- Pese também **interpretabilidade**, custo e velocidade.

O melhor não é o de maior número, é o que melhor **serve ao objetivo** e às restrições.

MÓDULO 04

Aprendizado Não-Supervisionado

Encontrar estrutura em dados sem rótulos: agrupamento, regras de associação e redução de dimensionalidade.

04

Encontrar estrutura sem rótulos

Não há resposta certa (y). O objetivo é **descobrir a estrutura** escondida nos dados: grupos, relações e eixos de variação.

Agrupamento

Juntar o que é **parecido** em grupos (clusters).

Associação

Descobrir **o que ocorre junto** (regras).

Redução de dim.

Resumir muitas variáveis em **poucos eixos**.

Onde se usa

Segmentação de clientes

Agrupar perfis de compra para personalizar marketing.

Sistemas de recomendação

"Quem comprou isto também levou aquilo."

Detecção de anomalias

O que não se encaixa em nenhum grupo: fraude, falhas.

Visualização e compressão

Reduzir dimensões para enxergar e armazenar melhor.

k-Means: agrupar em torno de centros



- Escolhe-se **k** centróides (◆). Cada ponto vai ao centróide **mais próximo**.
- O centróide é a **média** dos pontos do seu grupo.

Objetivo: grupos **compactos** , minimizando a distância dentro de cada cluster (inércia).

O algoritmo, passo a passo

1

Inicializar

Posiciona k centróides (em geral aleatoriamente).

2

Atribuir

Cada ponto vai ao centróide mais próximo.

3

Atualizar

Cada centróide vira a média dos seus pontos.

4

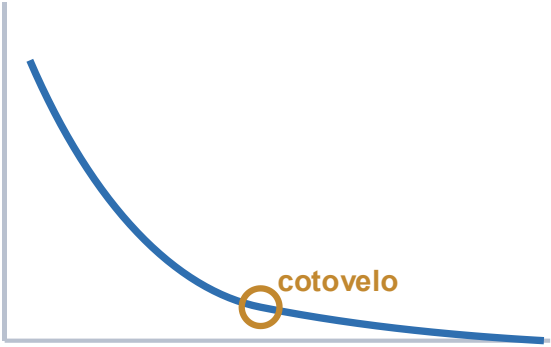
Repetir

Volta ao passo 2 até os grupos estabilizarem.

Converge rápido, mas o resultado **depende da inicialização**; roda-se várias vezes (k-means++) e fica-se com o melhor.

Escolher k: cotovelo e silhueta

Método do cotovelo



A inércia cai; a **dobra** indica o k onde parar.

Silhueta

Mede, para cada ponto, quão **perto do próprio grupo** e **longe dos outros** ele está.

≈ 1
bem agrupado

≈ 0
na fronteira

< 0
grupo errado

Escolhe-se o k de **maior silhueta média**.

Como as regras são descobertas

1

Contar frequentes

Mantém os itens com **suporte** acima de um mínimo.

2

Crescer combinações

Junta itens frequentes, descartando conjuntos raros.

3

Gerar regras

De cada conjunto frequente, forma regras $A \rightarrow B$.

4

Filtrar

Mantém só as regras fortes por **confiança** e **lift**.

Princípio Apriori: se um conjunto é raro, qualquer superconjunto também é — isso poda drasticamente a busca.

Suporte

Com que **frequência** A e B aparecem juntos no total de compras.

Confiança

Dado que levou A, qual a **probabilidade** de levar B também.

Lift

Quão **mais provável** que o acaso. > 1 : associação real.

Cesta de compras: fralda → cerveja

REGRA

 **Fralda** →  **Cerveja**

5% **60%** **3,0**
suporte confiança lift

- **5%** das compras têm fralda e cerveja.
- Quem leva fralda, em **60%** das vezes leva cerveja.
- **Lift 3**: 3× mais provável que um cliente qualquer.

Insight acionável: **posicionar os produtos próximos** ou cruzar promoções.

Como avaliar sem rótulos

Métricas internas

Silhueta e **inércia** medem coesão e separação dos grupos.

Estabilidade

Os grupos se mantêm com **amostras ou sementes** diferentes?

Validação do domínio

Os grupos **fazem sentido** para o especialista do negócio?

Sem gabarito, a **interpretação humana** é parte essencial da avaliação, mais do que no supervisionado.

MÓDULO 05

Redes Neurais e Deep Learning

Do neurônio artificial às redes profundas, CNNs, Transformers e o que esses modelos modernos nos permitem fazer.

05

O neurônio artificial

Recebe entradas, faz uma **soma ponderada** e passa por uma **função de ativação**. Aprender é ajustar os **pesos**.



Pesos (w)

A importância de cada entrada. **São eles que a rede aprende.**

Soma ponderada + viés

Combina as entradas em um único número.

Função de ativação

Introduz **não-linearidade** (ReLU, sigmoide) — o que permite aprender padrões complexos.

Um único neurônio é praticamente uma **regressão**. A força vem de **combinar milhares deles** em camadas.

Da unidade à rede

Neurônios organizados em **camadas**. Cada camada aprende uma representação mais abstrata; a rede ajusta os pesos por **retropropagação**.



Retropropagação: a rede prevê, mede o erro e ajusta os pesos para trás — repetindo milhares de vezes até acertar.

O que é Deep Learning

Redes neurais com **muitas camadas**. A grande virada: a rede **aprende sozinha quais atributos importam**, em vez de nós construí-los à mão.

ML CLÁSSICO

Nós desenhamos os atributos

dado bruto → **feature engineering manual** → modelo

Funciona bem com dados **tabulares** e poucos dados. Depende da nossa intuição sobre o problema.

DEEP LEARNING

A rede aprende os atributos

dado bruto → **representações aprendidas** → resposta

Brilha com **imagem, áudio e texto**. Em troca, exige **muitos dados** e **GPUs**.

Dados em escala

Hardware (GPU)

Melhores algoritmos

o que destravou o DL

CNNs — redes convolucionais

Feitas para **imagens**. Deslizam **filtros** que detectam padrões locais e os combinam em uma **hierarquia visual** — da borda ao objeto.

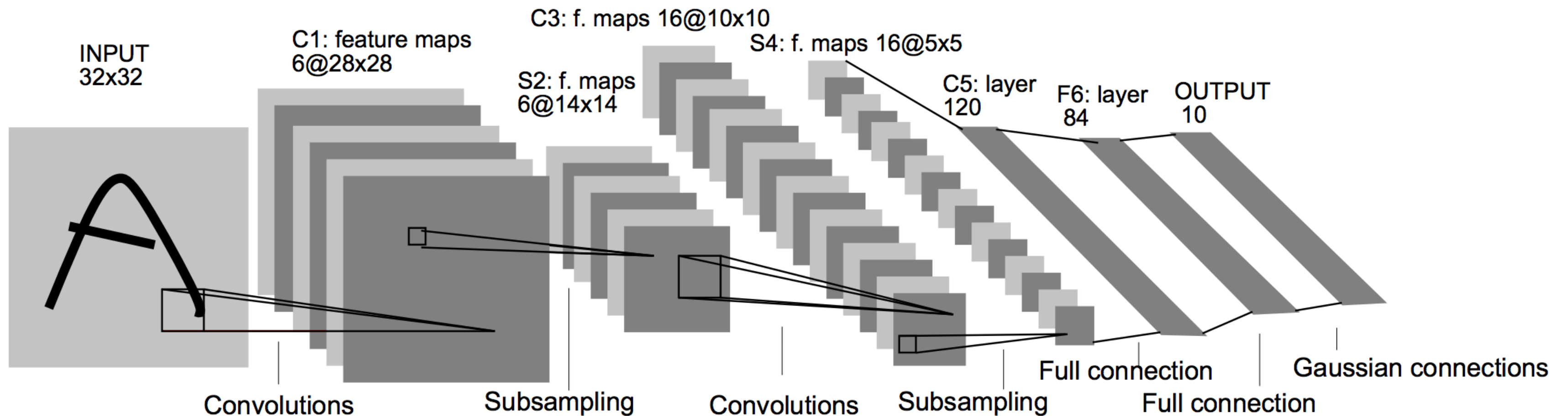


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

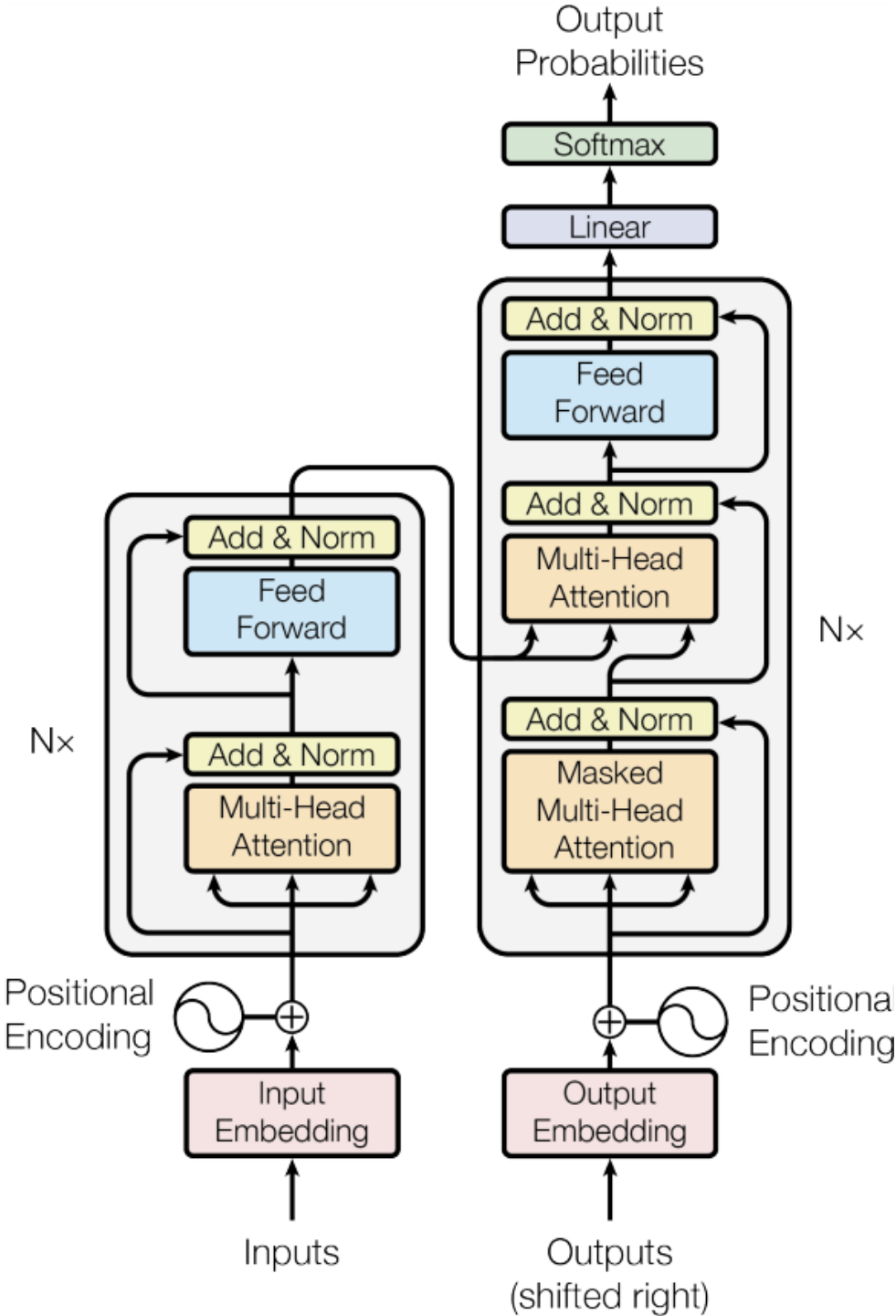
Transformers — o poder da atenção

Criados para **texto** (2017), hoje dominam quase tudo. A chave é a **atenção**: cada palavra olha para todas as outras e decide o que é relevante.

ATENÇÃO EM AÇÃO

O **banco** da **praça** está cheio

Ao ler "**banco**", o modelo dá mais peso a "**praça**", e entende que não é a instituição financeira. É assim que ele captura **contexto**.



O que levar para casa

01 **Supervisionado:** aprende de exemplos rotulados; cuidado com o equilíbrio viés-variância.

02 **Avaliar fora do treino,** com a métrica certa, separa o bom modelo do ilusório.

03 **Não-supervisionado** revela estrutura, grupos, regras e eixos, quando não há rótulos.

04 **Deep Learning** permite trabalhar com dados não estruturados

Cinco ideias para levar do módulo

- 01** **Mineração é um processo, não um algoritmo.** Entender › preparar › modelar › avaliar, e iterar (KDD / CRISP-DM).
- 02** **O valor está na preparação.** Coletar, limpar e transformar os dados é a maior parte do esforço.
- 03** **Conheça os dados antes de modelar.** Tipos, distribuições e correlações guiam a técnica.
- 04** **A técnica certa depende da pergunta.** Classificar, prever, agrupar ou associar.
- 05** **Avaliar fora do treino, com a métrica certa,** e ler o resultado no negócio é o que separa o modelo confiável do ilusório.

Mineração de dados aplicada aos negócios

Quatro princípios para levar do curso **para dentro da empresa**.

Comece pela pergunta

Parta da **decisão de negócio** , não dos dados nem da ferramenta. Sem pergunta clara, não há padrão útil.

Dado é ativo

Qualidade, **governança** e respeito à **LGPD** vêm antes do algoritmo. Lixo entra, lixo sai.

Do padrão à decisão

O modelo só vale se **muda uma ação** : o que produzir, a quem ofertar, onde agir.

Comece simples, itere

Um modelo **simples em produção** vale mais que um sofisticado na gaveta. Entregue cedo e melhore.

ENCERRAMENTO DO DIA 2 · PRÁTICA

Mãos à obra no Orange Data Mining

Sem escrever uma linha de código: montamos, treinamos e avaliamos modelos num fluxo visual de widgets.

O FLUXO QUE VAMOS CONSTRUIR



Obrigado

Perguntas? A seguir, mãos à obra com o Orange.